

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Dominik Uršič

**Avtomatizirano pridobivanje in
relacijsko-grafna analiza podatkov
omrežne infrastrukture National Grid**

DIPLOMSKO DELO

UNIVERZITETNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

MENTOR: izr. prof. dr. Matjaž Kukar

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Dominik Uršič

Naslov: Hibridni pristop k avtomatiziranemu pridobivanju in grafovski analizi podatkov omrežne infrastrukture National Grid

Vrsta naloge: Diplomaska naloga na univerzitetnem programu prve stopnje Računalništvo in informatika

Mentor: izr. prof. dr. Matjaž Kukar

Opis:

Diplomska naloga predstavlja razvoj avtomatiziranega ETL-sistema za pridobivanje in obdelavo podatkov o električni infrastrukturi operaterja National Grid v Združenem kraljestvu. Sistem avtomatizirano prenaša javno dostopne Excelove datoteke, jih arhivira v Google Cloud Storage ter procesira s pythonovimi skriptami. Obdelani podatki se naložijo v PostgreSQL bazo z Apache AGE grafovsko razširitvijo, kar omogoča hibridne relacijske in grafske analize omrežne strukture. Celoten proces je avtomatiziran z Google Cloud Scheduler ter vključuje mehanizme validacije, nadzora kakovosti podatkov in sledljivosti. Naloga demonstrira praktično uporabnost sistema z analitičnimi poizvedbami za optimizacijo omrežnih povezav in izračun prenosnih izgub, izvedenimi v cypherju in SQL-jezikih.

Title: A hybrid approach to automated data acquisition and graph-based analysis of National Grid network infrastructure

Description:

The thesis presents the development of an automated ETL system for acquiring and processing electrical infrastructure data from the National Grid operator in the United Kingdom. The system automatically downloads publicly available Excel files, archives them in Google Cloud Storage, and processes them using Python scripts. The processed data is loaded into a PostgreSQL database with Apache AGE graph extension, enabling hybrid relational and graph-based analyses of network structure. The entire process is automated using Google Cloud Scheduler and includes mechanisms for validation, data quality control, and traceability. The thesis demonstrates the practical applicability of the system through analytical queries for network connection optimization and transmission loss calculations, implemented in both SQL and Cypher languages.

Zahvaljujem se mentorju izr. prof. dr. Matjažu Kukarju za strokovno vodenje in podporo pri izdelavi diplomske naloge. Posebna zahvala gre tudi družini in prijateljem za razumevanje in spodbudo v času študija.

Kazalo

Povzetek

Abstract

1	Uvod	1
1.1	Motivacija	1
1.2	Opredelitev problema	2
1.3	Namen in cilji naloge	3
1.4	Pričakovane koristi in deležniki	4
1.5	Struktura dokumenta	5
2	Teoretično ozadje in pregled področja	7
2.1	Zgodovina spletnega strganja podatkov in izbira orodja	7
2.2	Javna električna infrastruktura v Združenem kraljestvu	9
2.3	National Grid API	9
2.4	Spletno strganje podatkov	10
2.5	Selenium WebDriver	10
2.6	Podatkovni cevovodi	11
2.7	Računalništvo v oblaku	12
2.8	Relacijske baze podatkov	12
2.9	PostgreSQL in delo z grafi	13
3	Metodologija in zasnova sistema	15
3.1	Identifikacija vira podatkov	15

3.2	Arhitektura sistema	16
3.3	Izbira orodij	17
3.4	Kontrola kakovosti podatkov	19
3.5	Podatkovne baze in pogoni s podporo za grafe	20
3.6	Izbira tehnologije	23
4	Implementacija sistema	25
4.1	Koraki delovanja sistema	26
4.2	Pridobivanje in shranjevanje podatkov	40
4.3	Obdelava podatkov	41
4.4	Vizualizacija in analiza	44
4.5	Avtomatizacija in nadzor	54
4.6	Kontrola kakovosti podatkov	55
5	Rezultati in evalvacija	57
5.1	Merila uspešnosti	57
5.2	Način evalvacije	58
5.3	Kriteriji uspeha	58
5.4	Rezultati in evalvacija	59
6	Zaključek	67
6.1	Glavne ugotovitve	67
6.2	Možnosti nadaljnjega razvoja	68
	Literatura	71

Seznam uporabljenih kratic

kratica	angleško	slovensko
ETL	Extract, Transform, Load	izlušči, preoblikuj, naloži
GCP	Google Cloud Platform	platforma Google Cloud
GCS	Google Cloud Storage	shramba Google Cloud
NG	National Grid	državno omrežje
DNO	Distribution Network Operator	operater distribucijskega omrežja
GSP	Grid Supply Point	napajalna točka omrežja
BSP	Bulk Supply Point	glavna napajalna točka
PRIM	Primary Substation	primarna transformatorska postaja
AGE	Apache Graph Extension	razširitev Apache Graph
Scheduler	Scheduler	razporejevalnik opravil

Povzetek

Naslov: Avtomatizirano pridobivanje in relacijsko-grafna analiza podatkov omrežne infrastrukture National Grid

Avtor: Dominik Uršič

Cilj diplomske naloge je razvoj in implementacija avtomatiziranega ETL-sistema za pridobivanje, obdelavo in shranjevanje podatkov o javni električni infrastrukturi operaterja National Grid v Združenem kraljestvu, z implementacijo hibridnega relacijsko-grafovskega pristopa k analizi omrežne strukture. Sistem bo redno prenašal javno dostopne Excelove datoteke s spletne platforme National Grid, jih arhiviral v Google Cloud Storage za zagotavljanje zgodovinske sledljivosti ter jih procesiral s pythonovimi skriptami za čiščenje, validacijo in transformacijo podatkov. Posebna pozornost bo namenjena kazalniku razpoložljive kapacitete (angl. *Demand Headroom*), ki predstavlja razliko med zanesljivo nosilnostjo omrežnega elementa in njegovo pričakovano najvišjo obremenitvijo ter tako določa preostalo zmogljivost pred potrebnimi infrastrukturnimi nadgradnjami. Obdelani podatki bodo naloženi v PostgreSQL podatkovno bazo, razširjeno z Apache AGE grafovsko nadgradnjo, kar bo omogočalo izvajanje tako tradicionalnih SQL kot tudi grafovskih cypher-skih poizvedb nad isto podatkovno strukturo. Ta hibridni pristop bo demonstriran z implementacijo analitičnih poizvedb za optimizacijo omrežnih povezav in izračun prenosnih izgub električne energije, izvedenih v obeh pristopih za neposredno primerjavo njihovih prednosti in omejitev. Celoten proces bo v celoti avtomatiziran z uporabo Google Cloud Scheduler, ki bo

zagotavljal redno izvajanje, mehanizme nadzora kakovosti podatkov ter popolno sledljivost vseh operacij. Sistem je zasnovan skalabilno, kar omogoča enostavno razširitev na dodatne operaterje distribucijskih omrežij ter predstavlja osnovo za potencialni razvoj celovite nacionalne platforme spremljanja električne infrastrukture v podporo energetske tranziciji.

Ključne besede: avtomatizacija pridobivanja podatkov, grafovske baze podatkov, Apache AGE, elektroenergetska infrastruktura, ETL sistem, hibridna analiza.

Abstract

Title: Automated data acquisition and relational/graph-based analysis of National Grid network infrastructure

Author: Dominik Uršič

The objective of this thesis is the development and implementation of an automated ETL system for acquiring, processing, and storing data on public electrical infrastructure of the National Grid operator in the United Kingdom, with implementation of a hybrid relational-graph approach to network structure analysis. The system will regularly download publicly available Excel files from the National Grid platform, archive them in Google Cloud Storage to ensure historical traceability, and process them using Python scripts for data cleaning, validation, and transformation. Special attention will be given to the Demand Headroom indicator, which represents the difference between the reliable capacity of a network element and its expected peak load, thus determining the remaining capacity before infrastructural upgrades are required. The processed data will be loaded into a PostgreSQL database extended with the Apache AGE graph extension, enabling the execution of both traditional SQL and graph-based Cypher queries on the same data structure. This hybrid approach will be demonstrated through the implementation of analytical queries for network connection optimization and electrical transmission loss calculations, executed in both approaches for direct comparison of their advantages and limitations.

The entire process will be fully automated using Google Cloud Scheduler, which will ensure regular execution, data quality control mechanisms, and

complete traceability of all operations. The system is designed to be scalable, enabling easy extension to additional distribution network operators and providing a foundation for potential development of a comprehensive national platform for monitoring electrical infrastructure in support of energy transition.

Keywords: automated data acquisition, graph databases, Apache AGE, electrical grid infrastructure, ETL system, hybrid analysis.

Poglavje 1

Uvod

1.1 Motivacija

V sodobnem svetu predstavlja električno omrežje kritično infrastrukturo, ki omogoča delovanje industrije, gospodinjstev, transporta in digitalnih storitev. Z naraščajočo integracijo obnovljivih virov energije, elektrifikacijo transporta ter razvojem pametnih omrežij postaja zanesljiv dostop do ažurnih podatkov o stanju električne infrastrukture vse bolj ključen za učinkovito načrtovanje, analizo in sprejemanje strateških odločitev na energetskega področja. V Združenem kraljestvu je National Grid [11] eden vodilnih operaterjev distribucijskega elektroenergetskega omrežja, ki pokriva obsežna geografska območja, vključno z regijami Vzhodni Midlands, Zahodni Midlands, Južni Wales ter Jugozahodna Anglija. Kot ključni distribucijski operater (DNO) je National Grid odgovoren za vzdrževanje in upravljanje srednjena-petostnega omrežja na svojem območju ter zagotavlja stabilno in zanesljivo oskrbo z električno energijo za več milijonov gospodinjstev in podjetij.

National Grid redno objavlja podatke o stanju svoje infrastrukture v obliki javno dostopnih Excelovih datotek na svoji spletni platformi. Ti podatki vključujejo obsežne informacije o transformatorskih postajah, napetostnih nivojih, geografskih lokacijah ter ključnem kazalniku razpoložljive kapacitete, ki določa preostalo zmogljivost posameznih omrežnih elementov.

Kljub javni dostopnosti pa je njihovo ročno pridobivanje, organizacija in posodabljanje časovno zamudno, podvrženo človeški napaki ter ne omogoča zgodovinske sledljivosti sprememb v infrastrukturi. Pomanjkanje avtomatiziranih sistemov za redno zajemanje in procesiranje teh podatkov predstavlja oviro za različne deležnike v energetske sektorju. Razvijalci projektov obnovljivih virov energije potrebujejo ažurne informacije o razpoložljivih kapacitetah za optimalno lociranje sončnih elektrarn in vetrnih parkov. Načrtovalci polnilne infrastrukture za električna vozila morajo identificirati lokacije z zadostno omrežno kapaciteto za priključitev hitrih polnilnic. Energetska podjetja in svetovalne agencije pa potrebujejo celovit pregled nad stanjem omrežja za strateško načrtovanje investicij in storitev.

1.2 Opredelitev problema

Trenutno pridobivanje podatkov o električni infrastrukturi National Grid poteka pretežno ročno, kar vključuje mesečno ali kvartalno odpiranje spletnih strani, iskanje ustreznih datotek, njihov prenos ter ročno vnašanje v lokalne evidence ali analitična orodja. Ta proces običajno zahteva 2–4 ure kvalificiranega dela na mesec in je podvržen več ključnim težavam.

Prvič, odsotnost zgodovinske sledljivosti predstavlja pomembno omejitve. Spletna platforma National Grid prikazuje zgolj najnovejše verzije datotek brez arhiviranja predhodnih stanj, kar onemogoča analizo časovnih sprememb v razpoložljivih kapacitetah, prepoznavanje trendov obremenitev ali longitudinalne študije razvoja omrežja. To otežuje dolgoročno načrtovanje infrastrukturnih investicij ter identifikacijo sistematičnih vzorcev v rasti elektroenergetskih potreb.

Drugič, pomanjkanje centralizirane in strukturirane podatkovne baze onemogoča integracijo informacij iz različnih virov ter izvajanje kompleksnih analiz, ki bi zahtevale kombiniranje podatkov o omrežni infrastrukturi z drugimi relevantnimi viri, kot so demografski trendi, prostorski načrti, vremenske napovedi ali projekcije rasti obnovljivih virov energije.

Tretjič, ročno procesiranje podatkov ne zagotavlja konsistentne kakovosti, saj lahko različni uporabniki uporabljajo različne metodologije čiščenja in transformacije podatkov, kar otežuje primerjavo rezultatov ter sodelovanje med organizacijami.

Četrto, električna infrastruktura je inherentno omrežne narave s hierarhičnimi povezavami med različnimi napetostnimi nivoji (GSP $\rightarrow$ BSP $\rightarrow$ PRIM), vendar tradicionalne relacijske baze niso optimizirane za predstavitev in analizo takšnih grafovskih struktur. To otežuje izvajanje omrežnih analiz, kot so iskanje optimalnih poti, identifikacija kritičnih vozlišč ali simulacije kaskadnih izpadov.

1.3 Namen in cilji naloge

Primarni namen sistema je avtomatizirati zajem javno dostopnih podatkov National Grid Electricity Distribution ter zagotoviti standardizirano, časovno označeno in sledljivo kopijo podatkov za nadaljnje analize. Sistem zmanjšuje ročno delo ter možnost napak pri ročnem prenosu podatkov, hkrati pa uvaža verzioniranje za popolno sledljivost sprememb v infrastrukturi skozi čas. Glavni cilj naloge je implementacija robustnega ETL (Extract, Transform, Load) sistema, ki bo avtomatizirano prenašal javno dostopne Excelove datoteke s spletne platforme National Grid, jih arhiviral v Google Cloud Storage staging okolju za zagotavljanje zgodovinske sledljivosti ter jih procesiral s pythonovimi skriptami za čiščenje, validacijo in transformacijo v standardizirane formate.

Obdelani podatki bodo naloženi v PostgreSQL podatkovno bazo, razširjeno z Apache AGE grafovsko nadgradnjo, kar bo omogočalo hibridni pristop k analizi podatkov. Ta pristop kombinira prednosti relacijskih baz (učinkovitost, zrelost, SQL-ekosistem) s prednostmi grafovskih baz (intuitivno modeliranje omrežnih struktur, podpora za cypherske poizvedbe, omrežne algoritme). Posebna pozornost bo namenjena kazalniku razpoložljive kapacitete, ki predstavlja ključno informacijo za načrtovanje novih priključitev in infrastrukturalnih

nih nadgradenj. Sistem bo omogočal spremljanje časovnih sprememb tega kazalnika ter identifikacijo območij z nizko razpoložljivo kapaciteto, kar je kritičnega pomena za načrtovalce novih objektov elektroinfrastrukture, kot so lokacije električnih polnilnic.

Celoten proces bo v celoti avtomatiziran z uporabo Google Cloud Scheduler, ki bo zagotavljal redno urno izvajanje v poslovnem času, mehanizme za validacijo kakovosti podatkov, obveščanje o napakah ter popolno sledljivost vseh operacij preko strukturiranega beleženja dogodkov. Dodatni cilj naloge je demonstracija praktične uporabnosti hibridnega relacijsko-grafovskega pristopa z implementacijo dveh kompleksnih analiz: optimizacija dodelitev primarnih transformatorskih postaj za minimizacijo prenosnih razdalj ter izračun prenosnih izgub električne energije na različnih segmentih omrežja. Vsaka analiza bo izvedena z uporabo tako cypherskega kot SQL-pristopa, kar bo omogočilo neposredno primerjavo obeh metodologij z vidika berljivosti, zmogljivosti in praktične uporabnosti.

1.4 Pričakovane koristi in deležniki

Implementacija avtomatiziranega sistema za pridobivanje podatkov o električni infrastrukturi bo prinesla oprijemljive koristi različnim deležnikom v energetskega sektorja. Sistem bo zmanjšal časovne zahteve za pridobivanje podatkov s trenutnih 2-4 ur mesečnega ročnega dela na nekaj minut avtomatiziranega procesa, kar predstavlja neposreden prihranek delovnega časa kvalificiranih kadrov ter zmanjšanje možnosti človeških napak pri prenosu in obdelavi podatkov.

Sistem zagotavlja trdno analitično podlago za taktično in strateško odločanje v energetskega sektorja. Ključni deležniki sistema so energetska podjetja in razvijalci projektov, ki bodo pridobili takojšen dostop do ažurnih podatkov o razpoložljivih kapacitetah, kar bo omogočilo hitrejše in bolj informirane odločitve o lokacijah novih projektov. Svetovalna podjetja v energetskega sektorja bodo lahko svojim strankam ponudila natančnejše analize in hitreje

pripravila študije izvedljivosti, medtem ko bodo investitorji v obnovljive vire energije pridobili kritične informacije za oceno primernosti lokacij za solarne elektrarne, vetrne parke ali baterijske sisteme.

Operaterji omrežja in načrtovalci infrastrukture bodo lahko uporabljali sistem za prepoznavanje ozkih grl v omrežju, načrtovanje nadgradenj ter optimizacijo razporeditve novih transformatorskih postaj. Javne ustanove in regulatorji bodo imeli na voljo konsistenten vir podatkov za spremljanje razvoja elektroenergetskega sistema ter pripravo strateških smernic za energetske tranzicije. Raziskovalci in svetovalni strokovnjaki pa bodo lahko izvajali napredne analize trendov porabe, napovedovanje prihodnjih kapacitet ter simulacije različnih scenarijev razvoja omrežja.

Strukturirana zgodovinska baza podatkov bo omogočila napredno analitiko za prepoznavanje trendov porabe, napovedovanje prihodnjih kapacitet in identifikacijo kritičnih točk v omrežju, kar bo podprlo strateško načrtovanje investicij v energetske infrastrukture in pospešilo prehod na trajnostne vire energije. Dolgoročno bo sistem omogočil enostavno razširitev na dodatne distributerje električne energije po celotni Veliki Britaniji, saj bo vzpostavljena arhitektura zlahka prilagodljiva za integracijo podatkov iz UK Power Networks, Scottish Power Energy Networks in drugih operaterjev.

1.5 Struktura dokumenta

V drugem poglavju predstavimo teoretično ozadje s pregledom ETL-procesov, pristopov k avtomatizaciji pridobivanja podatkov ter grafovskih podatkovnih baz. Posebej obravnavamo Apache AGE razširitev za PostgreSQL in jezik cypher ter utemeljimo izbiro PostgreSQL kot primerne platforme za našo aplikacijo, kljub temu da gre za OLTP (sprotno obdelovanje transakcij) bazo in ne specializiran OLAP (spletno razčlenitveno obdelovanje) sistem. Čeprav je PostgreSQL prvotno zasnovan za transakcijsko procesiranje, njegove napredne zmogljivosti (kompleksne poizvedbe, CTE-strukture, okenska funkcija, JSON-podpora) ter predvsem Apache AGE grafovska razširitev omogočajo

učinkovito analitično delo na srednje velikih podatkovnih množicah, kakršne predstavljajo podatki o električni infrastrukturi. Odločitev za PostgreSQL namesto specializiranih OLAP-rešitev (kot so ClickHouse ali Druid) temelji na več dejavnikih: relativno majhen obseg podatkov (nekaj tisoč transformatorskih postaj), potreba po hibridnem relacijsko-grafovskem pristopu, ki ga specializirani OLAP-sistemi ne podpirajo, ter prednost enotne platforme za transakcijsko in analitično obdelavo brez potrebe po dodatni ETL-cevi med sistemoma. V tretjem poglavju opisujemo metodologijo in arhitekturo sistema, vključno s celotno strukturo ETL-cevovoda, odločitvami pri izbiri tehnologij ter zasnovo podatkovnega modela z grafovsko shemo. Predstavimo, kako smo kombinirali relacijski in grafovski pristop za optimalno modeliranje hierarhične strukture elektroenergetskega omrežja.

V četrtem poglavju podrobno predstavimo implementacijo posameznih komponent, od pythonovih skript za obdelavo podatkov, konfiguracije Google Cloud Storage in Cloud Scheduler, do strukture relacijskih tabel in grafovskega modela. Prikažemo, kako PostgreSQL z Apache AGE razširitvijo omogoča izvajanje kompleksnih analitičnih poizvedb, ki sicer spadajo v domeno OLAP-sistemov, vendar jih lahko učinkovito izvajamo tudi na OLTP-platformi pri obsegu podatkov, s katerimi operiramo. V petem poglavju predstavimo rezultate avtomatizacije, implementacije analitičnih poizvedb v cypherskih in SQL-pristopih, kvantitativno primerjavo obeh pristopov ter vizualizacije ugotovitev o optimizaciji omrežnih povezav in prenosnih izgubah. Demonstriramo, kako hibridni pristop omogoča tako transakcijsko vnašanje novih podatkov kot tudi analitične poizvedbe nad omrežno strukturo brez potrebe po ločenih sistemih.

V šestem poglavju povzamemo ključne ugotovitve, evalviramo doseganje zastavljenih ciljev ter opredelimo možnosti nadaljnjega razvoja sistema, vključno s potencialnimi migracijami na specializirane OLAP-platforme v primeru bistveno večjih podatkovnih množic ali potrebe po analitiki v realnem času.

Poglavje 2

Teoretično ozadje in pregled področja

2.1 Zgodovina spletnega strganja podatkov in izbira orodja

Avtomatizirano zajemanje podatkov s spletnih strani se je v zadnjih dvajsetih letih precej spremenilo, ker so se spremenile tudi same spletne strani. V začetkih interneta so bile strani večinoma statične v obliki HTML-dokumenta, zato je za pridobivanje podatkov zadostovala preprosta analiza izvirne kode. Takrat so bile v uporabi predvsem rešitve, ki so temeljile na regularnih izrazih in osnovnem razčlenjevanju HTML-strukture. Pomemben mejnik je predstavljal razvoj specializiranih knjižnic za delo s HTML-dokumenti. Leta 2004 je prišel Beautiful Soup [14], ki je spletno strganje naredil veliko bolj dostopno. Ta pythonova knjižnica je z intuitivno sintakso omogočila enostavno ekstrakcijo podatkov iz kompleksnih HTML-struktur, vendar je bila še vedno omejena na statično vsebino. V tem času so se podobne knjižnice razvijale tudi za druge programske jezike, recimo jsoup [7] za Javo.

Leta 2008 je Scrapy [16] prinesel nov pristop s svojo asinhrono arhitekturo, ki je omogočila učinkovito obdelavo velikega števila strani hkrati. To pythonovo ogrodje je postalo nekakšen standard za večje projekte spletnega strga-

nja, saj je omogočalo distribuirano delo, avtomatsko upravljanje s piškotki in sejami ter robustno obravnavo napak. Takrat pa se je začel tudi velik premik v spletnem razvoju. Z uvedbo AJAX-tehnologij in enostranskih aplikacij so spletne strani postale veliko bolj dinamične. Vsebina se je začela nalagati asinhrono, elementi so se generirali z JavaScript kodo, podatki pa so se pridobivali preko API-klicev šele po začetnem nalaganju strani. Tradicionalne metode strganja naenkrat niso več zadostovale.

Rešitev je prišla z orodji za avtomatizacijo brskalnikov. Selenium WebDriver [17], ki je bil sicer prvotno razvit za testiranje spletnih aplikacij, se je izkazal za odlično orodje tudi za strganje kompleksnih strani. Za razliko od prejšnjih pristopov upravlja Selenium pravi brskalnik – lahko izvaja JavaScript, čaka na dinamično naložene elemente, simulira klike in druge uporabniške interakcije ter se spopada s kompleksnimi navigacijskimi tokovi. Selenium deluje preko WebDriver protokola, ki ga je leta 2018 standardiziral W3C konzorcij. Ta protokol omogoča komunikacijo med programsko kodo in brskalnikom na način, ki deluje prek različnih brskalnikov (Chrome, Firefox, Safari, Edge) in operacijskih sistemov. Osnova je odjemalec – strežnik model, kjer aplikacija pošilja ukaze gonilniku brskalnika, ta pa jih izvaja in vrača rezultate.

Za projekt avtomatizacije National Grid platforme je bil Selenium najboljša izbira iz več razlogov. Platforma zahteva avtentikacijo preko prijavnega obrazca, uporablja dinamično nalaganje vsebine, vključuje interaktivne zemljevide in vizualizacije ter ima tudi določene zaščitne mehanizme proti avtomatizaciji. Zato smo uporabili še gonilnik za Chrome, ki ga sistemi za zaznavanje ne odkrijejo (angl. *undetected chromedriver*), saj ta z različnimi tehnikami (modifikacija navigator objekta, odstranjevanje WebDriver zastavic, simulacija realističnih vzorcev gibanja miške) poskrbi, da sistem ne zazna avtomatizacije. Prehod od Beautiful Soup preko Scrapy do Selenium tako zrcali razvoj spletnih tehnologij. Medtem ko enostavnejša orodja še vedno dobro služijo za statične strani, kompleksne moderne aplikacije zahtevajo polno simulacijo brskalnika. Selenium z zmožnostjo izvajanja JavaScript

kode, čakanja na asinhrono naložene elemente in simulacije uporabniških interakcij trenutno predstavlja najzmogljivejšo rešitev za avtomatizirano pridobivanje podatkov iz sodobnih spletnih platform.

2.2 Javna električna infrastruktura v Združenem kraljestvu

Združeno kraljestvo ima kompleksen sistem električne infrastrukture, kjer različni akterji upravljajo prenos in distribucijo električne energije. Država je razdeljena na 14 geografskih območij, za katera je odgovornih šest podjetij. National Grid je eden glavnih operaterjev prenosnega omrežja, ki povezuje elektrarne z distribucijskimi omrežji [10].

National Grid redno objavlja podatke o zmogljivostih omrežja, vključno s podatki o razpoložljivi kapaciteti, ki opisuje razpoložljivo kapaciteto omrežja za nove priključitve. Ti podatki so ključni za energetske analize, načrtovanje infrastrukture in sprejemanje poslovnih odločitev. [8]

2.3 National Grid API

API Connected podatkovni portal podjetja National Grid predstavlja standardiziran vmesnik za programski dostop do javno objavljenih energetskih in omrežnih podatkov. Kljub temu da pokriva podatkovno domeno, ki se vsebinsko delno prekriva z obravnavanim področjem naloge, sistem izkazuje ključne strukturne omejitve, ki onemogočajo njegovo učinkovito implementacijo v kontekstu obravnavanih podatkov. Fundamentalna omejitev izhaja iz vnaprej definirane arhitekture podatkovnih nizov, pri čemer ponudnik določa tako obseg kot strukturo razpoložljivih podatkov. API v svoji trenutni konfiguraciji ne omogoča fleksibilnega prilagajanja podatkovnih zahtevkov specifičnim potrebam posameznega primera uporabe, temveč zgolj izpostavlja že obstoječe podatkovne nize v fiksni obliki, kot so bili izvirno objavljeni.

2.4 Spletno strganje podatkov

Spletno strganje (angl. *web scraping*) je tehnika avtomatskega pridobivanja podatkov s spletnih strani [9]. V pythonu obstajajo različne knjižnice, uporabili bomo Selenium, ki omogoča programsko upravljanje spletnega brskalnika in interakcijo z dinamičnimi spletnimi stranmi. Selenium deluje tako, da simulira dejanja pravega uporabnika v brskalniku, lahko klikne gumbe, izpolni obrazce, počaka na nalaganje elementov in izvozi podatke. Proces se tipično začne z inicializacijo brskalnika (v našem primeru Chromium z undetected-chromedriver za izogibanje detekciji avtomatizacije), nato pa skript sistematično navigira po spletni strani. Najprej se izvede prijava z vnosom uporabniškega imena in gesla, sledi navigacija do želenega dela aplikacije, kjer se sproži izvoz podatkov. Posebni funkciji (`WebDriverWait` in `expected_conditions`) zagotavljata, da skript počaka na popolno nalaganje elementov, preden z njimi upravlja, kar preprečuje napake zaradi asinhronega nalaganja vsebine. Ko je datoteka prenesena, se avtomatsko preimenuje s časovnim žigom in shrani v določeno mapo za nadaljnjo obdelavo. Pri spletnem strganju je pomembno upoštevati etične in pravne vidike, vključno s spoštovanjem `robots.txt` datotek, omejitev frekvence zahtev in pogojev uporabe spletnih strani [3]. V našem primeru gre za pridobivanje javno dostopnih podatkov z uporabo legitimnih prijavnih podatkov, kar zagotavlja skladnost s pogoji uporabe National Grid platforme.

2.5 Selenium WebDriver

Selenium WebDriver je odprtokodno orodje za avtomatizacijo spletnih brskalnikov, ki omogoča programsko interakcijo s spletnimi stranmi. Temelji na arhitekturi tipa odjemalec–strežnik: aplikacija pošilja ukaze brskalniku prek protokola WebDriver, brskalnikov gonilnik pa ta navodila izvede in vrne rezultat. Ko pythonova skripta pokliče Seleniumovo metodo, se ta pretvori v HTTP-zahtevo, ki jo gonilnik brskalnika (na primer ChromeDriver za Google Chrome) izvede neposredno v uporabniškem vmesniku. Selenium omogoča

različne načine za iskanje elementov na spletni strani, kot so uporaba identifikatorjev, CSS-selektorjev, izrazov XPath ali imen razredov. Z uporabo mehanizmov WebDriverWait in expected conditions lahko skripta eksplicitno počaka, da se določen pogoj izpolni (na primer, da element postane klikljiv), preden nadaljuje z izvajanjem. Tak pristop je ključen pri dinamičnih spletnih straneh, kjer se vsebina nalaga asinhrono prek JavaScripta.

2.6 Podatkovni cevovodi

Podatkovni cevovodi (angl. *data pipelines*) so avtomatizirani procesi za prenos podatkov od virov do končnih destinacij z možnostjo transformacije med potjo [19]. Tradicionalni ETL (Extract, Transform, Load) pristop se v zadnjem času vse bolj nadomešča z ELT (Extract, Load, Transform) pristopom, ki omogoča večjo fleksibilnost pri obdelavi podatkov. Ključna razlika med pristopoma je v zaporedju operacij. Pri ETL-pristopu se podatki najprej ekstrahirajo iz vira, nato transformirajo v vmesnem okolju in na koncu naložijo v ciljno podatkovno bazo. Nasprotno pa ELT-pristop najprej naloži surove podatke neposredno v podatkovno bazo, kjer se transformacije izvajajo z uporabo SQL-poizvedb in drugih orodij znotraj samega RDBMS-sistema. V našem sistemu implementiramo klasični ETL-pristop, ki se je izkazal za najbolj primerne glede na naravo podatkov in zahteve sistema. Uporaba ETL-pristopa pozitivno vpliva na zmogljivost RDBMS-sistema, saj PostgreSQL prejme le čiste, validirane podatke, kar zmanjšuje potrebo po kompleksnih SQL-transformacijah in s tem obremenitev podatkovne baze. To omogoča, da se PostgreSQL osredotoči na svoje primarne naloge, učinkovito shranjevanje, indeksiranje in serviranje podatkov končnim uporabnikom. Manjša obremenitev baze pomeni hitrejše odzivne čase pri poizvedbah, nižjo porabo sistemskih virov in večjo skalabilnost sistema. Dodatno ETL-pristop omogoča lažje odkrivanje in reševanje napak v podatkih, saj se te obravnavajo še pred vnosom v produkcijsko bazo, kar zagotavlja večjo integriteto podatkov in zanesljivost celotnega sistema.

2.7 Računalništvo v oblaku

Google Cloud Platform (GCP) ponuja različne storitve za delo s podatki, vključno z Google Cloud Storage za shranjevanje datotek in Google Cloud Scheduler za avtomatizirano izvajanje opravil [6]. Te storitve omogočajo skalabilno in zanesljivo infrastrukturo za podatkovne cevovode.

2.8 Relacijske baze podatkov

PostgreSQL je zmogljiva odprtokodna objektno-relacijska podatkovna baza, ki se pogosto uporablja za shranjevanje strukturiranih podatkov v podatkovnih aplikacijah [12]. Omogoča kompleksne poizvedbe, ACID-transakcije, različne razširitve za specifične potrebe ter napredno indeksiranje. Za naš sistem avtomatiziranega pridobivanja podatkov o električni infrastrukturi so ključne funkcionalne zahteve PostgreSQL sistema naslednje: podpora za velike količine časovnih serij podatkov (preko 1300 zapisov, seveda lahko PostgreSQL obdeli še veliko več zapisov), zmožnost hitrega vstavljanja novih podatkov preko operacij paketnega vnašanja, učinkovito indeksiranje na polju o razpoložljivih kapacitetah za hitre poizvedbe, podpora za JSON-podatkovne tipe za shranjevanje semistrukturiranih metapodatkov, ter zmožnost izvajanja kompleksnih analitičnih poizvedb z različnimi funkcijami. Sistem mora zagotavljati tudi verzioniranje podatkov, kjer se ohranjajo vse zgodovinske verzije za revizijske sledi in analizo trendov. Pomembna je tudi konfiguracija avtomatskega vzdrževanja preko autovacuum procesa, ki zagotavlja optimalno zmogljivost tudi pri velikem številu **UPDATE** in **DELETE** operacij. Dodatno mora sistem podpirati replikacijo za visoko razpoložljivost, omogočati *point in time recovery* za zaščito pred izgubo podatkov ter imeti nastavljeno redno varnostno kopiranje (angl. *pg dump*) vsaj enkrat dnevno. PostgreSQL razširitve, kot so *pg cronza* avtomatizirane naloge znotraj baze in *timescaledb* za optimizirano delo s časovnimi serijami, dodatno izboljšajo funkcionalnost sistema za naše specifične potrebe pri upravljanju podatkov.

2.9 PostgreSQL in delo z grafi

Poleg klasičnih relacijskih podatkovnih modelov se v sodobnih podatkovnih sistemih vse pogostejše pojavlja potreba po obdelavi grafovskih struktur, kjer so podatki predstavljeni kot vozlišča in povezave med njimi. Takšen pristop je posebej primeren za modeliranje omrežij, kot so energetska infrastruktura, prometna omrežja ali socialne mreže, kjer relacije med entitetami nosijo enako pomembno informacijo kot same entitete [1].

Čeprav PostgreSQL primarno sodi med relacijske podatkovne baze, omogoča učinkovito delo z grafi na več različnih načinov. Osnovni pristop temelji na modeliranju grafovskih struktur z uporabo relacijskih tabel, kjer se vozlišča hranijo v eni tabeli, povezave pa v drugi tabeli z referencami (tujimi ključi) na izvirno in ciljno vozlišče. Takšen model omogoča uporabo standardnih SQL-poizvedb za osnovne grafovske operacije, kot so iskanje sosedov, stopnje vozlišč in enostavne poti [4].

Za zahtevnejše grafovske analize ponuja PostgreSQL podporo z razširitvami. Ena najpomembnejših je *Apache AGE*, ki razširja PostgreSQL z lastnostmi večmodelne baze podatkov in dodaja podporo za grafni podatkovni model ter poizvedovalni jezik *openCypher*. *Apache AGE* omogoča izvajanje kompleksnih grafovskih poizvedb, kot so iskanje najkrajših poti, detekcija povezanih komponent in analiza omrežnih vzorcev, neposredno znotraj PostgreSQL okolja, brez potrebe po ločeni grafni bazi podatkov.

Alternativni pristop predstavlja razširitev *pgRouting*, ki je specializirana za delo z grafi v prostorskih podatkih in se pogosto uporablja v kombinaciji z razširitvijo *PostGIS*. Čeprav je primarno namenjena prometnim in geografskim omrežjem, se lahko njeni algoritmi za iskanje poti (Dijkstra, A*, Bellman-Ford) uporabijo tudi za analizo energetskih ali infrastrukturnih omrežij, kjer so povezave utežene z zmogljivostmi ali obremenitvami.

V kontekstu električne infrastrukture je grafovski model posebej primeren za predstavitev prenosnega in distribucijskega omrežja. Vozlišča lahko predstavljajo transformatorske postaje, razdelilne točke ali geografska območja, povezave pa fizične ali logične povezave med njimi. Atributi povezav, kot so

maksimalna zmogljivost, trenutna obremenitev ali razpoložljiv *podatek o kapacitetah*, omogočajo izvajanje analitičnih poizvedb, ki podpirajo odločanje pri načrtovanju novih priključitev ali nadgradenj omrežja.

Uporaba PostgreSQL za delo z grafi prinaša pomembno prednost v obliki enotne podatkovne platforme. Relacijski, časovni in grafovski podatki so shranjeni v istem sistemu, kar poenostavi arhitekturo, zmanjša operativne stroške in omogoča kombiniranje klasičnih SQL-poizvedb z grafovskimi analizami. Takšen hibridni pristop je posebej primeren za sisteme, kjer grafovske analize dopolnjujejo obstoječe relacijske in časovne podatkovne modele.

Poglavje 3

Metodologija in zasnova sistema

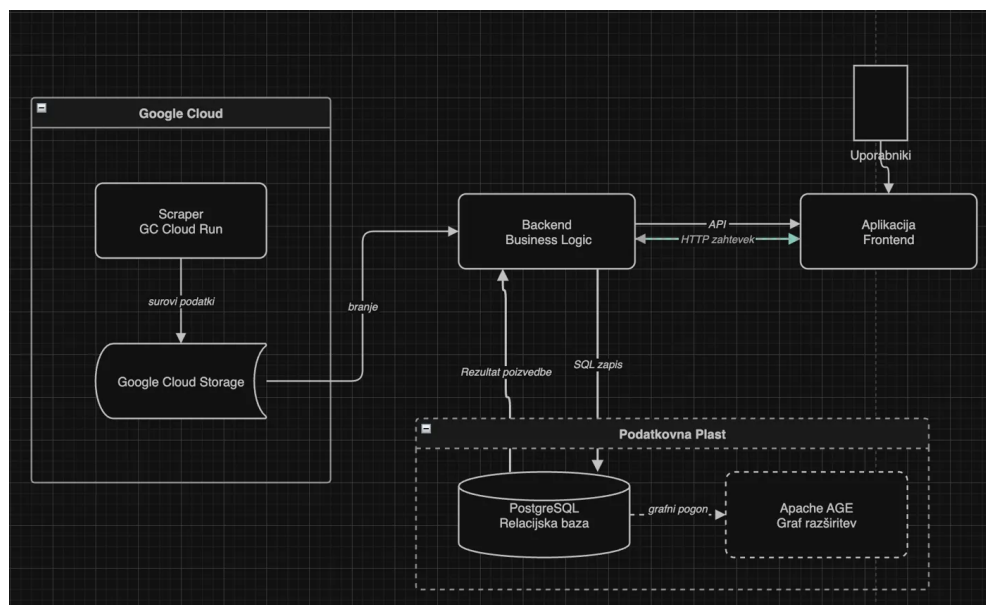
3.1 Identifikacija vira podatkov

Glavni vir podatkov je spletna stran National Grid, kjer so objavljene Excelove datoteke z informacijami o zmogljivostih omrežja. URL za dostop do podatkov je <https://www.nationalgrid.co.uk/our-network/network-capacity-map-application>.

Datoteke vsebujejo nabor tehničnih parametrov električne infrastrukture, strukturiranih v CSV-formatu. Ključna polja vključujejo ime postaje (angl. *Substation Name*), vrsto postaje (angl. *Asset Type*) ter koordinate lokacij (angl. *Latitude in Longitude*) za lažje geografsko pozicioniranje. Posebno pozornost namenjamo polju "razpoložljiva kapaciteta" (MW) (angl. *Demand Headroom*), ki predstavlja razpoložljivo kapaciteto za nove priključitve in je zelo pomemben parameter za razvijalce projektov pri ocenjevanju izvedljivosti novih povezav. Dodatni tehnični parametri vključujejo najvišjo obremenitev (angl. *Peak Demand*) in identifikator električnega omrežja (angl. *Network Reference ID*), ki omogočajo enolično identifikacijo vsake lokacije v nacionalnem omrežju.

3.2 Arhitektura sistema

Sistem bo implementiran po ETL-principu, pri čemer bomo dodali še vmesno (angl. *staging*) fazo v oblaku za večjo zanesljivost in sledljivost procesov. [18]



Slika 3.1: Prikaz arhitekture sistema

V prvi fazi ekstrakcije uporabljamo pythonove skripte s Selenium avtomatizacijo, ki se povežejo na spletno stran National Grid in prenesejo CSV-datoteke. Ta del deluje skoraj kot pravi uporabnik, saj simulira klike, čaka na nalaganje dinamičnih elementov in uporablja vse interaktivne komponente na strani. Ker gre za kompleksno spletno aplikacijo z JavaScript generiranimi elementi, je ta pristop nujen. Surovi podatki nato ne gredo direktno v bazo, ampak jih najprej pošljemo v Google Cloud Storage, kjer poteka vmesna transformacija. V tej fazi shranjevanja vmesnih podatkov (angl. *staging*) naredimo osnovno validacijo podatkov, počistimo manjkajoče vrednosti in standardiziramo formate. Te delno transformirane podatke shranimo kot vmesne datoteke, kar nam omogoča, da se lahko kadarkoli vrnemo nazaj in pogledamo, kako so podatki izgledali v določenem trenutku. To je zelo

koristno, če naletimo na kakšne težave ali potrebujemo ponovno procesiranje. Šele nato pride na vrsto končna transformacija, ki jo izvajamo s Pandas knjižnico. Ko je vse pripravljeno, podatke naložimo v PostgreSQL podatkovno bazo, kjer so nato na voljo za analizo in uporabo. Ta pristop z vmesno staging fazo v GCP nam zagotavlja večjo zanesljivost celotnega sistema. Če kdaj pride do napake v katerikoli fazi, imamo vedno shranjene vmesne rezultate in lahko proces ponovno poženemo od točke, kjer se je nekaj zalomilo. To je še posebej pomembno pri avtomatiziranih sistemih, kjer ni vedno nekoga, ki bi takoj opazil težavo. Celoten proces bo seveda avtomatiziran, uporabili pa bomo Google Cloud Scheduler, ki bo skripte zaganjal periodično po vnaprej določenem urniku. Tako bo sistem deloval samostojno in podatki se bodo osvežili brez kakršnegakoli ročnega posredovanja.

3.3 Izbira orodij

Za implementacijo sistema smo izbrali python 3.11, predvsem zaradi njegovega bogatega ekosistema knjižnic in odlične podpore za avtomatizacijo. Python se je v zadnjih letih uveljavil kot eden vodilnih jezikov za delo s podatki, kar se odraža tudi v razpoložljivosti kvalitetnih orodij za naše potrebe. Pri izbiri knjižnic smo kombinirali preverjene standardne rešitve s prilagojenimi komponentami. Za interakcijo z brskalnikom uporabljamo Selenium v kombinaciji z *undetected-chromedriver*, ki nam omogoča upravljanje z brskalnikom na način, da se izognemo detekciji avtomatizacije. To je zelo pomembno, saj večina sodobnih spletnih platform implementira zaščitne mehanizme proti avtomatiziranim dostopom. [2] Za delo s podatki se zanašamo na Pandas, ki je praktično postal standard za branje, transformacijo in obdelavo tabelarnih podatkov v pythonu. Ta knjižnica nam omogoča učinkovito delo s CSV-datotekami in izvajanje kompleksnih transformacij podatkov. Za komunikacijo z Google Cloud Storage uporabljamo uradno *google-cloud-storage* knjižnico, ki poskrbi za vso interakcijo s oblacnim shranjevanjem in upravljanje *staging* datotek. Za beleženje vseh dogodkov in napak uporabljamo

vgrajeni modul za beleženje (angl. *logging modul*), ki nam omogoča strukturirano spremljanje delovanja sistema. Poleg standardnih knjižnic smo razvili tudi nekaj prilagojenih komponent. **AbstractScriptRunner** je abstraktni razred, ki standardizira izvajanje ETL-skript in vključuje vgrajeno logiko za elegantno obravnavo napak. Tako vse naše skripte sledijo enakemu vzorcu izvajanja, kar olajša vzdrževanje in razumevanje kode. Centralizirano konfiguracijo celotnega sistema upravljamo preko konfiguracijskega (angl. *config*) modula, ki vključuje nastavitve za logiranje, povezavo s GCS-klientom in podatkovne direktorije. Tako imamo vse nastavitve na enem mestu, kar občutno olajša prilagajanje sistema različnim okoljem. Razvili smo tudi *gduutil* (Generic Dataset Utilities), nabor prilagojenih funkcij za delo z geografskimi podatkovnimi nizi, ki rešujejo specifične izzive našega projekta. Selenium smo izbrali kot orodje za avtomatizirano pridobivanje podatkov, ker National Grid platforma uporablja dinamično generiran spletni vmesnik, kjer se vsebina nalaga preko JavaScript kode in ni dostopna v surovi HTML-strukturi strani. Tradicionalni pristopi spletnega strganja s knjižnicami, kot sta *requests* ali BeautifulSoup, ne bi bili zadostni, saj ne morejo izvajati JavaScript kode in posledično ne morejo dostopati do dinamično generiranih elementov. Selenium simulira vedenje pravega uporabnika v brskalniku, kar omogoča interakcijo s kompleksnimi spletnimi aplikacijami, čakanje na nalaganje elementov ter avtomatsko klikanje gumbov in izpolnjevanje obrazcev, kar je ključno za dostop do Excelovih datotek na National Grid platformi.

3.3.1 Infrastruktura

Celoten sistem temelji na kombinaciji oblačnih storitev, kar nam omogoča fleksibilnost pri shranjevanju in obdelavi podatkov. Za shranjevanje surovih in delno transformiranih datotek uporabljamo Google Cloud Storage. To je objektno shranjevanje, ki ga ponuja Google Cloud Platform in se je izkazalo za idealno rešitev za naš *staging layer*. Tukaj se shranjujejo CSV-datoteke, ki jih sistem prenese iz National Grid platforme, še preden jih procesiramo in naložimo v bazo. Prednost GCS je v tem, da nam omogoča praktično

”neomejeno shranjevanje po relativno nizki ceni, hkrati pa so podatki vedno dostopni in zanesljivo shranjeni. Poleg tega lahko kadarkoli dostopamo do zgodovinskih verzij datotek, če potrebujemo ponovno procesiranje ali analizo, kako so se podatki spreminjali skozi čas. Za končno shranjevanje strukturiranih, obdelanih podatkov pa uporabljamo PostgreSQL podatkovno bazo. PostgreSQL smo izbrali zaradi njegove robustnosti, odlične podpore za kompleksne poizvedbe in geografske podatke preko PostGIS razširitve. Gre za relacijsko bazo, ki omogoča učinkovito indeksiranje in iskanje po podatkih, kar je ključno za kasnejšo analizo in vizualizacijo. Za avtomatizacijo celotnega procesa skrbi Google Cloud Scheduler. To je periodična (angl. *cron*) storitev v oblaku, ki omogoča zanesljivo periodično izvajanje naših skript. Nastavimo lahko natančne urnike, kdaj naj se sistem zažene (vsak dan ob določeni uri ali vsak teden na določen dan). Cloud Scheduler je zanesljiv, ne zahteva vzdrževanja strežnika, ki bi moral biti vedno prižgan, in nam pošlje obvestila, če pride do napak pri izvajanju. Tako je celoten ETL-proces popolnoma avtomatiziran in deluje brez potrebe po ročnem posredovanju. V primeru spremembe strukture podatkov *staging* arhitektura v Google Cloud Storage zagotavlja, da so izvirne datoteke ohranjene v nespremenjenem stanju, kar omogoča analizo sprememb in prilagoditev transformacijskih skript brez izgube podatkov. Modularna zasnova pythonovih skript omogoča hitro prilagoditev mapiranja stolpcev in validacijskih pravil.

3.4 Kontrola kakovosti podatkov

Za zagotavljanje zanesljivosti sistema in kakovosti podatkov implementiramo več nivojev preverjanj, ki bodo našli potencialne težave že v zgodnjih fazah procesiranja. Validacija podatkovnih tipov je ključna za pravilno delovanje celotnega sistema. Ta bo namreč preveril, ali so numerične vrednosti res številke, ali so datumi v pravilnem formatu in ali besedilna polja ne vsebujejo nepričakovanih znakov ali vsebin. Če naleti na vrednosti, ki ne ustrezajo pričakovanemu tipu, bo zabeležil opozorilo in se odločil, ali lahko vrednost

pretvori ali jo mora zavrniti. Posebno pozornost bomo namenili tudi preverjanju in upravljanju manjkajočih vrednosti. Pri nekritičnih poljih lahko manjkajoče vrednosti nadomestimo s privzetimi vrednostmi ali jih pustimo prazne, medtem ko bodo pri kritičnih poljih manjkajoče vrednosti povzročile zavrnitev celotnega vnosa.

```
1 gdf[ network_reference_id ] = gdf[ network_reference_id
   ].astype(int)
2 gdf[ parent_id ] = gdf[ parent_id ].apply(
3     lambda x: int(x) if not pd.isna(x) else None
4 )
5
6 gdf[ in_v ] = gdf[ voltage_str ].apply(
7     lambda x: None
8     if pd.isna(x)
9     else float(x.split( / )[0]) * 1000
10    if / in x
11    else float(x)
12 )
13
14 gdf.drop_duplicates(subset=[ network_reference_id ],
    inplace=True)
```

Listing 3.1: Čiščenje in transformacija podatkov

3.5 Podatkovne baze in pogoni s podporo za grafe

Z naraščajočo kompleksnostjo podatkov in potrebo po učinkovitem modeliranju odnosov med entitetami so grafno usmerjene podatkovne rešitve postale pomemben del sodobnih podatkovnih arhitektur. Sistemi, ki podpirajo grafe, omogočajo učinkovite poizvedbe v več korakih, analizo omrežij in modeliranje kompleksnih relacij, kar je pogosto neučinkovito ali neizvedljivo v klasičnih

relacijskih bazah. V nadaljevanju so predstavljeni izbrani sistemi, ki bodisi delujejo kot namenske grafne baze bodisi kot razširitve ali poizvedovalni pogoni nad obstoječimi podatkovnimi viri.

3.5.1 PuppyGraph

PuppyGraph ni klasična grafna baza podatkov, temveč *grafni poizvedovalni pogon*, ki omogoča izvajanje grafnih poizvedb neposredno nad obstoječimi podatkovnimi viri, kot so relacijske baze, podatkovna skladišča ali podatkovna jezera. Ključna značilnost sistema je arhitektura brez ETL-procesov, saj podatkov ni treba kopirati ali transformirati v ločeno grafno shrambo.

PuppyGraph logično preslika obstoječe podatke v grafni model in podpira standardne grafne poizvedovalne jezike, kot sta openCypher in gremlin. Tak pristop omogoča hitro uvedbo grafnih analiz v obstoječe sisteme, pri čemer se ohranijo prednosti primarnega podatkovnega vira, kot so konsistentnost, varnost in upravljanje podatkov.

3.5.2 AgensGraph

AgensGraph je grafna baza podatkov, ki temelji na relacijski bazi PostgreSQL. Gre za hibridni sistem, ki združuje relacijski in grafni podatkovni model znotraj enotnega podatkovnega strežnika. Uporabnikom omogoča, da v isti podatkovni bazi kombinirajo klasične SQL-poizvedbe in grafne operacije.

Zaradi izpeljave iz PostgreSQL podeduje AgensGraph transakcijski model ACID, zanesljivost ter bogat ekosistem orodij. Takšna zasnova je primerna za primere uporabe, kjer je potrebno tesno prepletanje relacijskih in grafnih podatkov brez uvajanja dodatne infrastrukture.

3.5.3 RedisGraph

RedisGraph je modul za Redis, ki implementira lastnostni grafni model v pomnilniku. Namenjen je predvsem scenarijem, kjer je ključnega pomena nizka latenca in visoka hitrost poizvedb. Sistem uporablja openCypher kot

poizvedovalni jezik ter interno predstavlja graf z uporabo redkih matrik in linearno-algebrskih operacij.

Zaradi *in-memory* narave je RedisGraph posebej primeren za operativne in realnočasovne grafne primere uporabe, kot so priporočilni sistemi ali analiza omrežij v živo. Slabost pristopa je večja poraba pomnilnika in omejena primernost za zelo velike, trajne grafe.

3.5.4 Cayley

Cayley je odprtokodna grafna baza podatkov, prvotno zasnovana po vzoru grafnega sistema Freebase. Poseben poudarek namenja podpori za povezane podatke in ogrodja za opis virov (RDF-standarde).

Sistem podpira več poizvedovalnih jezikov in lahko deluje nad različnimi hrambnimi mehanizmi, vključno z relacijskimi in ključ – vrednost bazami. Zaradi svoje prilagodljivosti je Cayley primeren za raziskovalne namene, semantične grafe in znanstvene aplikacije, kjer igra interoperabilnost podatkov pomembno vlogo.

3.5.5 Apache AGE

Apache AGE (*A Graph Extension*) je razširitev za PostgreSQL, ki tej relacijski bazi doda podporo za grafni podatkovni model. Namesto ločene grafne baze AGE omogoča shranjevanje vozlišč in povezav znotraj PostgreSQL ter poizvedovanje z uporabo openCypher jezika.

Tak pristop omogoča hibridno uporabo SQL in grafnih poizvedb nad enotnim podatkovnim skladiščem. Apache AGE je še posebej primeren za organizacije, ki že uporabljajo PostgreSQL in želijo grafne funkcionalnosti dodati brez večjih arhitekturnih sprememb.

3.6 Izbira tehnologije

Glede na to, da je v obravnavanem primeru podatkovna baza že vzpostavljena v sistemu PostgreSQL, se kot najprimernejša izbira izkaže Apache AGE. Razširitev omogoča enostavno integracijo grafnega podatkovnega modela neposredno v obstoječo bazo, brez potrebe po migraciji podatkov ali uvajanju ločenega grafnega strežnika. Apache AGE podpira poizvedovalni jezik openCypher, ki je postal standard za delo z lastnostnimi grafi, kar poenostavi izražanje kompleksnih relacijskih vzorcev in večkorakovnih poizvedb. Poleg tega AGE izkorišča preverjen transakcijski mehanizem PostgreSQL, zagotavlja ACID-lastnosti ter omogoča sočasno uporabo SQL in grafnih poizvedb nad istimi podatki. Integracija AGE v PostgreSQL poteka preko namestitve razširitve, ki se aktivira z ukazom `LOAD 'age'`. Grafovski podatki se shranjujejo v posebnih PostgreSQL shemah, kjer vsak graf dobi svojo shemo (npr. `grid_network`), znotraj katere se ustvarijo tabele za vozlišča in relacije. Ključna tehnična značilnost je uporaba proprieternega podatkovnega tipa `agtype`, ki služi kot ovojnica za JSON-like strukture vozlišč in relacij ter omogoča kompatibilnost s PostgreSQL sistemskimi tipi podatkov.

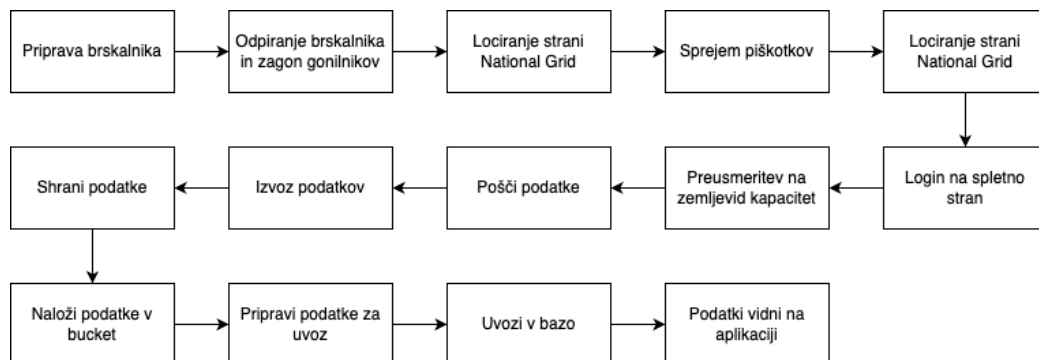
Izmenjava podatkov med SQL in cypher zahteva eksplicitne pretvorbe tipov. Rezultati poizvedb v cypherju se vrnejo kot `agtype`, ki jih je treba kastirati v native PostgreSQL tipe z uporabo zaporedja pretvorb (npr. `(value)::text::numeric`). Hibridne poizvedbe se tipično strukturirajo kot Common Table Expressions (CTE), kjer se poizvedba v cypherju izvede v prvem CTE-bloku za grafovsko navigacijo, njeni rezultati pa se nato pretvorijo in uporabijo v navadnih SQL-operacijah za numerično procesiranje.

Z vidika zmogljivosti AGE predstavlja kompromis med fleksibilnostjo grafovskih operacij ter zmogljivostjo specializiranih sistemov. Ker deluje kot plast nad PostgreSQL, vključuje dodatno delo za razčlenjevanje cypherjeve sintakse, prevajanje v PostgreSQL operacije ter pretvorbe med `agtype` in nativnimi tipi. To lahko povzroči nižjo zmogljivost v primerjavi z namenskimi grafovskimi bazami, še posebej pri globokih pregledih. Vendar pa je za

omrežja srednje velikosti, kakršno predstavlja električna infrastruktura National Grid (nekaj tisoč vozlišč, nekaj deset tisoč relacij), ta presežek sprejemljiv. PostgreSQL B-tree in GiST indekse je mogoče uporabiti tudi za grafovske tabele, kar omogoča učinkovito iskanje ter kombinacijo s PostGIS razširitvijo za geografsko indeksiranje. Takšna hibridna zasnova zmanjšuje arhitekturno kompleksnost sistema, poenostavi vzdrževanje in omogoča postopno uvajanje grafnih pristopov v obstoječe relacijsko okolje, pri čemer se izognemo potrebi po ETL-procesih med relacijsko in grafovsko bazo ter s tem povezanim težavam sinhronizacije podatkov, redundance ter dodatne kompleksnosti arhitekture.

Poglavje 4

Implementacija sistema



Slika 4.1: Postopek za strganje podatkov, odlaganje v oblak, obdelavo ter uvoz v bazo.

4.1 Koraki delovanja sistema

4.1.1 Priprava brskalnika in zagon gonilnikov

Prvi korak implementacije vključuje konfiguracijo Selenium WebDriver z uporabo razreda Options. Sistem inicializira Chrome brskalnik v *headless* načinu, pri čemer so dodani argumenti `--no-sandbox`, `--disable-dev-shm-usage` in `--window-size=3840,2160`. Argument `--no-sandbox` omogoča delovanje brskalnika brez Chromovega izolirnega izvajanja (angl. *sandboxing*) mehanizma, kar je pogosto nujno v kontejneriziranih okoljih, kjer lahko sandbox povzroča konflikt z omejenimi sistemskimi privilegiji. Argument `--disable-dev-shm-usage` preusmeri uporabo deljenega pomnilnika z `/dev/shm` na disk, kar preprečuje napake v okoljih z omejenim ali premajhnim deljenim pomnilnikom (npr. Docker). Določitev velikosti okna zagotavlja pravilno renderiranje strani tudi v headless načinu. Brskalnik se inicializira preko `webdriver.Chrome(options=options)`, medtem ko `WebDriverWait` skrbi za zanesljivo upravljanje nalaganja dinamičnih elementov.

```
1 from selenium.webdriver.chrome.options import Options
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import
    expected_conditions as EC
5 from google.cloud import storage
6
7 options = Options()
8 options.add_argument( --headless )
9 options.add_argument( --no-sandbox )
10 options.add_argument( --disable-dev-shm-usage )
11 options.add_argument( --window-size=3840,2160 )
12
13 driver = webdriver.Chrome(options=options)
```

Listing 4.1: Priprava brskalnika

4.1.2 Odpiranje brskalnika in zagon gonilnikov

Ko je brskalnik inicializiran, se izvede navigacija na glavni portal National Grid (<https://www.nationalgrid.co.uk/network-opportunity-map/>). Sistem počaka 5 sekund za popolno nalaganje strani, skripta pa uporablja čakanje z metodo *.sleep()* za zagotovitev, da so vsi elementi pripravljeni za interakcijo.

```
1 driver.get( https://www.nationalgrid.co.uk/network-  
2           opportunity-map/ )  
           time.sleep(5)
```

Listing 4.2: Odpiranje strani

4.1.3 Lociranje strani National Grid

Po uspešni inicializaciji sistem poišče in klikne na povezavo za prijavo, ki vodi na prijavni portal. Skripta uporablja CSS-selektorje za natančno lociranje elementov, pri čemer se navigacijska logika prilagaja morebitnim spremembam v strukturi strani. Sistem beleži vsak korak navigacije v dnevniško datoteko za kasnejšo analizo in odpravljanje težav.

4.1.4 Sprejem piškotkov

Upravljanje s piškotki je izvedeno neposredno z iskanjem gumba za sprejem vseh opsijskih piškotkov. Sistem z uporabo `WebDriverWait` in pogoja `element_to_be_clickable` poišče element *Accept all optional cookies* ter ga, ko je dostopen, klikne. Tak pristop zagotavlja, da se klik izvede šele, ko je gumb dejansko interaktiven. Ker koda ne vključuje dodatnega preverjanja ali bloka za obravnavo izjem (angl. *try-except*) bloka, se predpostavlja, da je pasica (angl. *banner*) vedno prisotna; v primeru manjkajočega elementa bi se sprožila izjema.

```
1 cookie_btn = wait.until(EC.element_to_be_clickable(  
2     (By.CSS_SELECTOR, 'a[title = Accept all optional  
3     cookies ]'))  
4 cookie_btn.click()
```

Listing 4.3: Potrjevanje piškotkov

4.1.5 Lociranje strani National Grid (navigacija)

Navigacija do podatkovnega portala poteka v več korakih. Po prijavi sistem navigira na specifični URL zemljevida kapacitet, sistem pa počaka na popolno nalaganje aplikacije, preden nadaljuje z naslednjimi koraki. Blokirati je treba nalaganje zemljevida in vseh povezav, ki se na njem prikazujejo, saj se v nasprotnem primeru okno brskalnika poruši.

```
1 driver.execute_cdp_cmd( Network.enable , {})  
2 driver.execute_cdp_cmd( Network.setBlockedURLs , {  
3     urls : [  
4         *mapbox.com/* ,  
5         *tiles.mapbox.com/* ,  
6         *tile.openstreetmap.org/* ,  
7         *tilelayer* ,  
8         *VectorTile* ,  
9         *features* ,  
10        *geojson*  
11    ]  
12 })
```

Listing 4.4: Potrjevanje piškotkov

4.1.6 Prijava na spletno stran

Elektronski naslov in geslo se vneseta v ustrezna polja z ID-ji

customer-portal-form-field_emailAddress in customer-portal-form-field_password. Skripta uporablja JavaScript executor za zanesljiv klik na prijavni gumb, kar obvladuje tudi primere, ko standardni Selenium klik ne deluje. Po prijavi sistem počaka in preveri URL za potrditev uspešne prijave. Pred vsakim vpisovanjem v polje "email" ali "password" pokličemo metodo .clear(), da je polje zagotovo prazno.

```
1 login_btn = wait.until(EC.element_to_be_clickable((By.  
    CSS_SELECTOR, 'a[href^=/customer-portal/login ]')))  
2 driver.execute_script( arguments[0].click(); , login_btn  
    )  
3 time.sleep(3)  
4  
5 email_input = wait.until(EC.  
    visibility_of_element_located((By.ID, customer-  
        portal-form-field__emailAddress )))  
6 password_input = wait.until(EC.  
    visibility_of_element_located((By.ID, customer-  
        portal-form-field__password )))  
7  
8 email_input.clear()  
9 email_input.send_keys(EMAIL)  
10 password_input.clear()  
11 password_input.send_keys(PASSWORD)
```

Listing 4.5: Login

4.1.7 Preusmeritev na zemljevid kapacitet

Po uspešni prijavi se izvede navigacija na aplikacijo zemljevida kapacitet. Tukaj je dejanski zemljevid blokirán, saj ga *headless browser* ne more naložiti. Sistem najprej sprejme pogoje uporabe s klikom na čósent checkbox”in potrditvenim gumbom. Nato odpre levi navigacijski panel in klikne na zavihek ”Data”, kjer so dostopni izvozni podatki. Vsak korak vključuje preverjanje prisotnosti elementov ter ustrezno obravnavo napak.

```
1 driver.get( https://www.nationalgrid.co.uk/our-  
    network/network-capacity-map-application )  
2 wait.until(lambda d: d.execute_script( return  
    document.readyState ) == complete )
```

Listing 4.6: Lociranje zemljevida

4.1.8 Pošči podatke

Izvoz podatkov se sproži preko postopnega odpiranja ustreznih uporabniških vmesnikov. Najprej sistem s pomočjo objekta `WebDriverWait` počaka, da je element z identifikatorjem `data-pill` ključen za nadaljevanje interakcije, nakar se klik izvede preko `execute_script`, kar zagotavlja zanesljivo aktivacijo tudi v primerih, ko standardni klik ni zadosten. Sledi aktivacija gumba za odprtje stranske vrstice, izbranega preko CSS-selektorja. Ko je stranska vrstica uspešno odprta, sistem poišče oznako `Data` z uporabo XPATH-izraza. Pred klikanjem se element premakne v vidno območje z uporabo `scrollIntoView(true)`, kar zagotavlja zanesljivo interakcijo tudi v *headless* načinu. S tem je uporabniški vmesnik ustrezno pripravljen za nadaljnje korake izvoza podatkov.

```
1 data_button = wait.until(EC.element_to_be_clickable((By.  
    ID, data-pill )))  
2 driver.execute_script( arguments[0].click(); ,  
    data_button)  
3 open_sidebar_btn = wait.until(EC.element_to_be_clickable  
    ((  
4         By.CSS_SELECTOR ,  
5         button.btn.btn--continue.btn--default.btn--  
            small  
6     )))  
7     open_sidebar_btn.click()  
8  
9 data_label = wait.until(EC.element_to_be_clickable(  
10     (By.XPATH, //label[contains(text(), 'Data')  
        ] )))  
11     driver.execute_script( arguments[0].  
        scrollIntoView(true); , data_label)  
12     data_label.click()
```

Listing 4.7: Podatki

4.1.9 Izvoz podatkov

Prenos CSV-datoteke se sproži s klikom na gumb za izvoz, izbran preko CSS-selektorja. Klik na gumb se izvede preko `execute_script`, kar omogoča zanesljivo sproženje dogodka tudi v primerih, ko standardni klik ni zadosten. Po sprožitvi izvoza sistem z uporabo `WebDriverWait` preveri, da element z razredom `btn--loading` izgine, kar označuje konec procesa generiranja CSV-datoteke. Ta mehanizem omogoča deterministično zaznavanje zaključka izvoza.

```
1 export_button = wait.until(EC.element_to_be_clickable(  
2     (By.CSS_SELECTOR, button.btn.btn--primary.  
3         export-button )))  
4 driver.execute_script( arguments[0].scrollIntoView(  
5     true); , export_button)  
6 time.sleep(0.5)  
7 driver.execute_script( arguments[0].click(); ,  
8     export_button)  
9  
10 WebDriverWait(driver, 30).until_not(  
11     EC.presence_of_element_located(  
12         (By.CSS_SELECTOR, button.btn.btn--primary.  
13             export-button.btn--loading )  
14     )  
15 )
```

Listing 4.8: Izvoz

4.1.10 Shrani podatke in naloži podatke v vedro

Podatke je za nadaljno uporabo treba shraniti v "Google Cloud Storage bucket". Datoteke se za lažje upravljanje organizira v mapno strukturo po datumih (leto/mesec/dan). GCS zagotavlja verzioniranje, kar omogoča dostop do vseh zgodovinskih verzij podatkov.

```
1     timestamp = datetime.now(timezone.utc).strftime( %Y
2         -%m-%d_%H-%M-%S )
3     upload_to_gcs(
4         bucket_name= diplomska-461311_cloudbuild ,
5         source_file=full_path,
6         destination_blob= exports/
            wpd_network_capacity_map_{}.csv .format(
                timestamp)
    )
```

Listing 4.9: Nalaganje v GCS

4.1.11 Pripravi podatke za uvoz

V tej fazi se izvede transformacija podatkov s Pandas knjižnico. CSV-datoteka se naloži v GeoDataFrame, kjer se izvedejo naslednje operacije: odstranjevanje praznih vrstic, standardizacija imen stolpcev, pretvorba podatkovnih tipov, validacija vrednosti *Demand Headroom* in pretvorba koordinat v standardni format. Dodajo se tudi metapodatki o času izvoza in verziji podatkov.


```
1 gdf = dl_reader.read_csv(  
2     cls.dl_all_path,  
3     crs= epsg:4326 ,  
4     xcol= Longitude ,  
5     ycol= Latitude ,  
6     use_saved=True,  
7 )  
8 columns = {  
9     Substation Name : name ,  
10    Bulk Supply Point Name : bsp ,  
11    Substation Number : xref ,  
12    Upstream Voltage : up ,  
13    Downstream Voltage : down ,  
14    Demand Headroom (MVA) : demand_headroom_mva ,  
15    Generation Headroom (MVA) :  
16        generation_headroom_mva ,  
17    Fault Level Headroom (kA) :  
18        fault_level_headroom_ka ,  
19 }  
20 gdf = gdf[list(columns.keys())]  
21 gdf.rename(columns=columns, inplace=True)  
22 gdf[ name ] = (  
23     gdf[ name ].str.extract( ^(.+?)(?=\s+\d| \(\d|\s-\s|  
24         $) ) [0].str.title()  
25 )  
26 gdf[ bsp ] = (  
27     gdf[ bsp ].str.extract( ^(.+?)(?=\s+\d| \(\d|\s-\s|  
28         $) ) [0].str.title()  
29 )  
30 gdf[ last_updated ] = str(cls.all_last_updated)  
31 gdf.drop_duplicates(inplace=True)
```

Listing 4.10: Priprava podatkov

4.1.12 Priprava baze

Proces priprave vključuje definicijo tabelne strukture, določitev primarnih ključev ter vzpostavitev indeksov za optimalno zmogljivost poizvedb.

Struktura tabele in primarni ključ

Tabela je zasnovana z avtomatsko generiranim primarnim ključem `id`, ki uporablja PostgreSQL sekvenco za dodeljevanje unikatnih identifikatorjev. Poleg tega vključuje dva geometrijska stolpca: `geometry` v koordinatnem sistemu WGS84 (EPSG:4326) za globalno kompatibilnost ter `geometry_projected` v britanskem nacionalnem koordinatnem sistemu OSGB36 (EPSG:27700) za natančne lokalne izračune razdalj. Stolpec `properties` tipa JSONB omogoča fleksibilno shranjevanje dodatnih atributov brez potrebe po spreminjanju tabelne sheme, medtem ko `xref` zagotavlja unikatno povezavo z zunanjimi identifikatorji.

```
1 CREATE TABLE IF NOT EXISTS uk_dataset. core/ng/  
  substation/bsp/v2025_10  
2 (  
3   id integer NOT NULL DEFAULT  
4     nextval('uk_dataset. core/ng/substation/bsp/  
      v2025_10_id_seq '::regclass),  
5   geometry geometry(Geometry,4326) NOT NULL,  
6   geometry_projected geometry(Geometry,27700) NOT NULL  
7   ,  
8   properties jsonb DEFAULT '{} '::jsonb,  
9   xref character varying COLLATE pg_catalog. default ,  
10  CONSTRAINT core/ng/substation/bsp/v2025_10_pkey  
    PRIMARY KEY (id),  
11  CONSTRAINT core/ng/substation/bsp/v2025_10_xref_key  
    UNIQUE (xref)  
12 )  
13 TABLESPACE pg_default;
```

```
14 ALTER TABLE IF EXISTS uk_dataset. core/ng/substation/bsp
    /v2025_10
15 OWNER to dataset_user;
```

Listing 4.11: Ustvarjanje tabelne strukture

Geografski indeksi

Za učinkovito izvajanje prostorskih poizvedb, kot so iskanje najbližjih transformatorskih postaj ali izračuni razdalj, so nujni specializirani geografski indeksi. PostgreSQL z razširitvijo PostGIS podpira GiST (Generalized Search Tree) indekse, ki so optimizirani za geometrijske podatke. Ustvarimo dva ločena GiST-indeksa: enega za `geometry` stolpec v globalnem koordinatnem sistemu in drugega za `geometry_projected` v britanskem nacionalnem sistemu. Ta dvojnost omogoča optimalne rezultate tako pri globalni vizualizaciji kot pri lokalnih izračunih, kjer je projekcija OSGB36 bistveno natančnejša za britansko geografsko območje.

```
1 CREATE INDEX IF NOT EXISTS core/ng/substation/bsp/
    v2025_10_geometry
2 ON uk_dataset. core/ng/substation/bsp/v2025_10
    USING gist
3 (geometry)
4 TABLESPACE pg_default;
5
6 CREATE INDEX IF NOT EXISTS core/ng/substation/bsp/
    v2025_10_geometry_projected
7 ON uk_dataset. core/ng/substation/bsp/v2025_10
    USING gist
8 (geometry_projected)
9 TABLESPACE pg_default;
```

Listing 4.12: Ustvarjanje geografskih indeksov

B-tree indeks za hitro iskanje

Poleg geografskih indeksov je ključnega pomena tudi učinkovito iskanje po `xref` stolpcu, ki vsebuje zunanje referenčne identifikatorje (npr. National Grid Reference ID). Za ta namen uporabljamo B-tree indeks, ki je optimiziran za iskanje po enakosti in razponskih poizvedbah nad skalarnimi vrednostmi. Ta indeks bistveno pospeši operacije, kjer je treba poiskati specifično transformatorsko postajo po njenem identifikatorju ali po tem ključu izvesti JOIN-operacije med različnimi tabelami.

```
1 CREATE INDEX IF NOT EXISTS core/ng/substation/bsp/  
  v2025_10_xref  
2 ON uk_dataset. core/ng/substation/bsp/v2025_10  
  USING btree  
3 (xref COLLATE pg_catalog. default ASC NULLS LAST)  
4 TABLESPACE pg_default;
```

Listing 4.13: Ustvarjanje B-tree indeksa

4.1.13 Uvoz v bazo

Transformirani podatki se iz `GeoDataFrame` strukture naložijo v PostgreSQL bazo preko `SQLAlchemy` povezave. Najprej se izvede posodobitev obstoječih zapisov, nato pa se kliče še funkcija `add_leaf_dataset()`, ki poskrbi za dejanski uvoz geografskih podatkov s pripadajočimi atributi v ustrezno tabelo.

Implementirana je transakcijska logika, ki zagotavlja atomskost operacij, bodisi se vsi podatki uspešno vnesejo bodisi se celotna transakcija razveljavi v primeru napake. Transakcijski kontekst se upravlja preko `SQLAlchemy connection` objekta, ki avtomatsko izvede `COMMIT` ob uspešnem zaključku ali `ROLLBACK` ob napaki.

```
1 for taxonomy in [ core.ng.substation.bsp , core.ng.  
  substation.pss ]:  
2     table_name = make_table_name(taxonomy)  
3     con.execute(
```

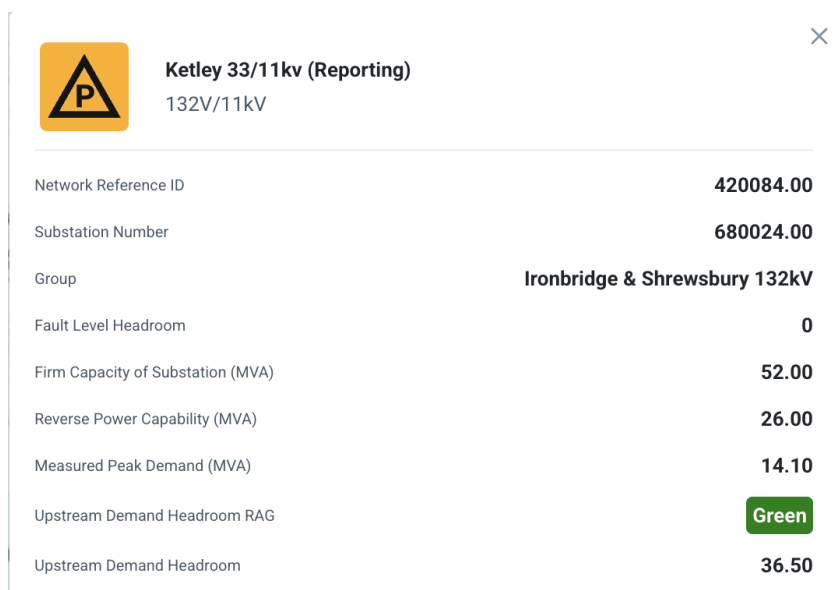
```
4
5      UPDATE dataset. {table_name}
6      SET properties = (properties - '
7          demand_headroom_mva') ||
8          jsonb_build_object('dhr',
9              (properties -> 'demand_headroom_mva')::
10                 float)
11      WHERE properties ->> 'demand_headroom_mva' IS
12          NOT NULL;

    )
add_leaf_dataset(taxonomy, con)
```

Listing 4.14: Uvoz v bazo

4.1.14 Podatki, vidni v aplikaciji

Podatki, ki so rezultat celotnega obdelovalnega procesa, so po zaključku vseh validacijskih in objavnih korakov neposredno vidni v aplikaciji. To pomeni, da se ob vsakem uspešnem zagonu sistema najnovejši, preverjeni in standardizirani podatki samodejno posodobijo v uporabniškem vmesniku, kjer jih lahko končni uporabniki takoj pregledajo, filtrirajo in uporabljajo za nadaljnje analize ali operativne odločitve. Dostopni so v realnem času, prek interaktivnih preglednic, kartografskih prikazov ali dinamičnih vizualizacij, odvisno od funkcionalnosti posamezne aplikacijske komponente. S tem se zagotavlja popolna transparentnost med procesom zajema podatkov in njihovo končno uporabo, saj aplikacija vedno prikazuje zadnjo potrjeno verzijo informacij, sinhronizirano z osrednjo bazo. Takšna integracija omogoča enoten vpogled v stanje omrežja, objektov in procesov, ne glede na izvor podatkov ali njihovo tehnično kompleksnost v ozadju.



		Ketley 33/11kv (Reporting) 132V/11kV	
Network Reference ID		420084.00	
Substation Number		680024.00	
Group		Ironbridge & Shrewsbury 132kV	
Fault Level Headroom		0	
Firm Capacity of Substation (MVA)		52.00	
Reverse Power Capability (MVA)		26.00	
Measured Peak Demand (MVA)		14.10	
Upstream Demand Headroom RAG		Green	
Upstream Demand Headroom		36.50	

Slika 4.2: Primer iz UI-aplikacije

4.2 Pridobivanje in shranjevanje podatkov

Proces pridobivanja in shranjevanja podatkov je zasnovan kot popolnoma avtomatiziran sistem, ki zagotavlja zanesljivo, ponovljivo in časovno sledljivo osveževanje informacij iz zunanjih virov.

Po uspešnem prenosu se datoteke avtomatsko naložijo v namensko vedro v oblaku, kjer se hranijo v strukturirani mapni hierarhiji po letu, mesecu in dnevu prenosa. Ta organizacija omogoča enostavno arhiviranje, hitro iskanje in učinkovito upravljanje zgodovinskih podatkovnih posnetkov. GCS-infrastruktura omogoča tudi vklop verzioniranja, kar pomeni, da se ob vsakem novem prenosu ohrani popolna zgodovina sprememb in tako se stare datoteke ne prepišejo, temveč ostanejo dostopne za primerjalne analize ali rekonstrukcijo prejšnjih stanj. [13]

4.3 Obdelava podatkov

4.3.1 Struktura vhodnih podatkov

National Grid objavlja podatke o električni infrastrukturi v obliki CSV-datoteke z obsežno strukturo 49 stolpcev, ki pokrivajo različne aspekte omrežnih elementov. Datoteke vsebujejo tri glavne kategorije transformatorskih postaj: BSP (Bulk Supply Point) za glavne napajalne točke srednje napetosti, Primary za primarne transformatorske postaje ter Generation za generacijske priključke najvišjih napetosti.

Ključni stolpci vključujejo identifikacijske podatke (**Network Reference ID**, **Parent Network Reference ID**), geografske koordinate (**Latitude**, **Longitude**), napetostne nivoje (**Upstream Voltage**, **Downstream Voltage**), kapacitetne parametre (**Firm Capacity of Substation**, **Measured Peak Demand**) ter ključni kazalnik razpoložljive kapacitete (**Demand Headroom**), ki predstavlja razliko med zanesljivo nosilnostjo in pričakovano najvišjo obremenitvijo postaje.

Tabela 4.1: Primer surovih podatkov za BSP-postajo

Stolpec	Vrednost
Network Reference ID	245518
Parent Network Reference ID	245518
Substation Name	Abergavenny Primary
Asset Type	BSP
Latitude	51.8396
Longitude	-3.0133
Upstream Voltage	132/66
Downstream Voltage	66
Firm Capacity (MVA)	60.0
Measured Peak Demand (MVA)	57.31
Demand Headroom (MVA)	2.69
Generation Headroom (MVA)	-197.47
Total Inferred Generation (MVA)	254.6664

Tabela 4.1 prikazuje primer surovih podatkov za BSP-transformatorsko postajo Abergavenny Primary, kjer je vidna delna struktura podatkov.

4.3.2 Transformacija in čiščenje podatkov

Za obdelavo prenesenih CSV-datotek smo razvili namensko prilagojeno pythonovo skripto, ki temelji na uporabi knjižnice `pandas` in `geopandas` za delo z geografskimi podatki. Skripta po vzpostavitvi povezave z Google Cloud Storage najprej prebere ustrezne datoteke, shranjene v oblaku, ter jih pretvori v `GeoDataFrame` strukturo za nadaljnjo obdelavo.

Prvi korak obdelave vključuje pretvorbo identifikacijskih stolpcev v ustrezne podatkovne tipe. Stolpec `Network Reference ID` se pretvori v celoštevilski tip, medtem ko se za `Parent Network Reference ID`, ki lahko vsebuje manjkajoče vrednosti pri BSP-postajah brez nadrejenega elementa, uporabi `lambda` funkcija za pogojno pretvorbo.

```
1 gdf[ network_reference_id ] = gdf[ network_reference_id
    ].astype(int)
2 gdf[ parent_id ] = gdf[ parent_id ].apply(
3     lambda x: int(x) if not pd.isna(x) else None
4 )
```

Listing 4.15: Pretvorba identifikacijskih stolpcev

Posebno pozornost zahteva obdelava napetostnih nivojev, saj uporablja National Grid različne formate zapisa. Napetosti so lahko zapisane kot enojna vrednost (npr. "11") ali kot razmerje med napetostjo navzgor in navzdol (npr. "132/66"). Implementirali smo funkcijo, ki razčleni oba formata in pretvori vrednosti v volte, pri čemer napetosti, zapisane v kilovoltih (kV), pomnoži s 1000.

```
1 gdf[ in_v ] = gdf[ voltage_str ].apply(
2     lambda x: None
3     if pd.isna(x)
4     else float(x.split( / )[0]) * 1000
```



```
5     if / in x
6     else float(x)
7 )
```

Listing 4.16: Razčlenjevanje napetostnih nivojev

4.3.3 Validacija in deduplikacija

Po transformaciji podatkov sledi sistematično čiščenje, ki vključuje odstranjevanje podvojenih zapisov. Ker lahko ena transformatorska postaja nastopa večkrat v različnih kontekstih (npr. kot BSP in kot Primary), izvajamo deduplikacijo na podlagi `network_reference_id` ključa, pri čemer ohranimo prvi (najbolj relevanten) zapis.

```
1 gdf.drop_duplicates(subset=[ network_reference_id ],
    inplace=True)
```

Listing 4.17: Odstranjevanje duplikatov

Sistem izvaja tudi validacijo podatkovnih tipov in preverjanje konsistentnosti vrednosti. Preverjamo, da so numerične vrednosti, kot so `Demand`, `Headroom`, `Firm Capacity` in geografske koordinate, v veljavnem obsegu. Manjkajoče vrednosti v kritičnih poljih (identifikatorji, koordinate) povzročijo zavrnitev celotnega zapisa, medtem ko se manjkajoče vrednosti v nekritičnih poljih (npr. opcijske opombe) ohranijo kot `NULL`.

4.3.4 Geografska obdelava

Pomemben del transformacije je vzpostavitev dvojne geografske reprezentacije. Izvorne WGS84 koordinate (EPSG:4326) iz stolpcev `Latitude` in `Longitude` se uporabijo za ustvarjenje prvega geometrijskega objekta, primerne za globalno vizualizacijo in kompatibilnost s spletnimi kartami. Hkrati se koordinate projicirajo v britanski nacionalni koordinatni sistem OSGB36 (EPSG:27700), ki omogoča natančne izračune razdalj in površin na britanskem ozemlju brez popačenj, ki jih vnašajo globalne projekcije.

4.3.5 Priprava za uvoz v bazo

Tako pripravljeni podatki se nato dopolnijo z dodatnimi metapodatki, vključno z datumom izvoza, verzijo podatkovne sheme in identifikacijsko oznako vira. Vsi dodatni atributi, ki niso del osnovne tabelne strukture, se serializirajo v JSONB-format in shranijo v stolpec `properties`, kar zagotavlja fleksibilnost pri dodajanju novih polj brez potrebe po spreminjanju podatkovne sheme.

Končni rezultat tega postopka je homogen in validiran `GeoDataFrame` podatkovni nabor, pripravljen za nadaljnje nalaganje v PostgreSQL podatkovno bazo, kjer se lahko uporablja tako za transakcijske operacije kot za kompleksne analitične poizvedbe.

4.4 Vizualizacija in analiza

Po uspešnem prenosu in strukturiranju podatkov v podatkovno bazo smo razvili nabor analitičnih poizvedb, ki omogočajo vpogled v strukturo in delovanje elektroenergetskega omrežja. Za vsako analizo smo implementirali dva pristopa: prvega s čistim SQL-jezikom, ki dela neposredno na relacijskih tabelah, in drugega s kombinacijo jezika cypher za grafovsko navigacijo ter SQL za numerične izračune. Ta primerjava omogoča ovrednotenje prednosti in slabosti obeh pristopov pri delu z omrežnimi infrastrukturnimi podatki.

4.4.1 Analiza prenosa električne energije in izgub

Ena ključnih analiz, ki smo jo izvedli, je ocena izgub električne energije pri prenosu skozi različne nivoje omrežja. Poizvedba sledi poti električne energije od Grid Supply Point (GSP) prek Bulk Supply Point (BSP) do končnih Primary Substation (PRIM) točk ter izračuna kumulativne izgube na posameznih segmentih.

SQL-pristop

Klasični SQL-pristop uporablja relacijske JOIN-operacije za povezovanje tabel `grid.prim`, `grid.bsp` in `grid.gsp` preko tujih ključev. Prvi del pridobi vse potrebne podatke o postajah in izračuna razdalje med njimi, drugi del določi ustrezne vrednosti upornosti glede na napetostne nivoje, zadnji del pa izvede končne izračune prenosnih izgub.

```
1 WITH power_paths AS (  
2     SELECT  
3         p.name AS prim_name ,  
4         g.gsp_name ,  
5         extract_max_voltage(g.gsp_name) AS  
6             gsp_voltage_kv ,  
7         b.bsp_name ,  
8         extract_max_voltage(b.bsp_name) AS  
9             bsp_voltage_kv ,  
10        calculate_distance(g.gsp_x, g.gsp_y ,  
11            b.bsp_x, b.bsp_y) AS  
12            gsp_bsp_distance_km ,  
13        calculate_distance(b.bsp_x, b.bsp_y ,  
14            p.x, p.y) AS  
15            bsp_prim_distance_km  
16 FROM grid.prim p  
17 JOIN grid.bsp b ON p.bsp_nrid = b.bsp_nrid  
18 JOIN grid.gsp g ON b.gsp_nrid = g.gsp_nrid  
19 ),  
20 loss_calculations AS (  
21     SELECT  
22         prim_name, gsp_name, bsp_name ,  
23         CASE  
24             WHEN gsp_voltage_kv >= 100 THEN 0.05  
25             WHEN gsp_voltage_kv >= 50 THEN 0.08  
26             ELSE 0.12
```

```
23     END as gsp_resistance_ohms_per_km ,
24     CASE
25         WHEN bsp_voltage_kv >= 100 THEN 0.05
26         WHEN bsp_voltage_kv >= 50 THEN 0.08
27         ELSE 0.12
28     END as bsp_resistance_ohms_per_km ,
29     gsp_bsp_distance_km, bsp_prim_distance_km ,
30     gsp_voltage_kv, bsp_voltage_kv
31 FROM power_paths
32 WHERE gsp_voltage_kv > 0 AND bsp_voltage_kv > 0
33 )
34 SELECT
35     prim_name, gsp_name, bsp_name ,
36     ROUND(calculate_transmission_loss_percent(
37         gsp_bsp_distance_km, gsp_voltage_kv ,
38         gsp_resistance_ohms_per_km
39     ), 4) as gsp_bsp_loss_percent ,
40     ROUND(calculate_transmission_loss_percent(
41         bsp_prim_distance_km, bsp_voltage_kv ,
42         bsp_resistance_ohms_per_km
43     ), 4) as bsp_prim_loss_percent
44 FROM loss_calculations
45 ORDER BY (gsp_bsp_loss_percent + bsp_prim_loss_percent)
46     DESC;
```

Listing 4.18: SQL pristop - jedro poizvedbe

SQL-pristop je neposreden in uporablja tradicionalne relacijske operacije. Povezovanje tabel poteka preko tujih ključev (`bsp_nrid`, `gsp_nrid`), kar zagotavlja učinkovitost pri ustrezno indeksiranih tabelah. Povprečen čas izvajanja poizvedbe na celotnem naboru podatkov znaša 8 sekund.

Cypherski pristop

Hibridni pristop uporablja Apache AGE razširitev za grafovsko navigacijo po omrežni strukturi ter SQL za numerične izračune. Ključna razlika je v uporabi vzorca MATCH, ki omogoča intuitivno opisovanje poti skozi omrežje preko relacij CONNECTED_TO_BSP in FEEDS_FROM.

```
1 WITH power_paths AS (  
2     SELECT * FROM cypher('grid_network', $$  
3         MATCH (p:Prim)-[:CONNECTED_TO_BSP]->(b:Bsp)  
4             -[:FEEDS_FROM]->(g:Gsp)  
5         RETURN  
6             p.name as prim_name,  
7             g.gsp_name as gsp_name,  
8             g.gsp_x as gsp_x, g.gsp_y as gsp_y,  
9             b.bsp_name as bsp_name,  
10            b.bsp_x as bsp_x, b.bsp_y as bsp_y,  
11            p.x as prim_x, p.y as prim_y  
12    $$) as (  
13        prim_name agtype, gsp_name agtype,  
14        gsp_x agtype, gsp_y agtype,  
15        bsp_name agtype, bsp_x agtype, bsp_y agtype,  
16        prim_x agtype, prim_y agtype  
17    )  
18 ),  
19 distance_calculations AS (  
20     SELECT  
21         prim_name::text as prim_name,  
22         gsp_name::text as gsp_name,  
23         bsp_name::text as bsp_name,  
24         extract_max_voltage(gsp_name::text) as  
25             gsp_voltage_kv,  
         extract_max_voltage(bsp_name::text) as  
             bsp_voltage_kv,
```

```

26     calculate_distance(
27         (gsp_x)::text::numeric, (gsp_y)::text::
           numeric,
28         (bsp_x)::text::numeric, (bsp_y)::text::
           numeric
29     ) as gsp_bsp_distance_km,
30     calculate_distance(
31         (bsp_x)::text::numeric, (bsp_y)::text::
           numeric,
32         (prim_x)::text::numeric, (prim_y)::text::
           numeric
33     ) as bsp_prim_distance_km
34 FROM power_paths
35 )

```

Listing 4.19: Cypher pristop - grafovska navigacija

Cypher-pristop jasno izraža domensko logiko elektroenergetskega sistema, kjer relacije `CONNECTED_TO_BSP` in `FEEDS_FROM` neposredno odražajo fizične povezave in smer toka energije. Cena te semantične jasnosti je nižja zmogljivost – povprečen čas izvajanja znaša 42 sekund, kar je približno petkrat dlje od SQL-pristopa. Ta razlika izhaja iz dodatne kompleksnosti grafovskega procesiranja ter večkratnih pretvorb podatkovnih tipov med Apache AGE agtype formatom in nativnimi PostgreSQL tipi.

Primerjava pristopov

Tabela 4.2 prikazuje povprečne čase izvajanja obeh pristopov na podlagi petih meritev.

Tabela 4.2: Primerjava zmogljivosti pristopov SQL in cypher		
Pristop	Povprečni čas (s)	Relativna razlika
SQL (relacijski)	8,0	1,0
Cypher (grafovski)	42,0	5,25

Kljub nižji zmogljivosti pa prinaša cypher pomembne prednosti pri vzdrževanju in razumevanju kode. Grafovska notacija je intuitivnejša za domensko specifične analize, kjer je pomembna navigacija po omrežni strukturi. Za produkcijske aplikacije z visokimi zahtevami po odzivnosti bi bil primernejši SQL-pristop, medtem ko je cypher uporaben za raziskovalne analize in prototipiranje, kjer je berljivost pomembnejša od surove hitrosti izvajanja.

Razlogi za nižjo zmogljivost cypherskega pristopa so večplastni in izhajajo iz arhitekture Apache AGE. Vsaka poizvedba v cypherju se mora najprej parsirati, validirati ter prevesti v ustrezne PostgreSQL operacije, kar vključuje dodaten presek v primerjavi z neposrednim SQL. Uporaba **agtype** podatkovnega tipa zahteva večkratne pretvorbe med JSONB-interno reprezentacijo in nativnimi PostgreSQL tipi, kar se kaže v sekvencah kot `(value)::text::numeric`. Vsaka taka pretvorba zahteva dodatno procesiranje, še posebej pri večjih rezultatnih množicah. Indeksiranje grafovskih struktur v AGE prav tako ni tako optimizirano kot SQL-indeksi. Čeprav je mogoče ustvariti indekse na **agtype** stolpcih, ti delujejo preko JSONB-indeksiranja, kar je manj učinkovito od direktnih B-tree ali GiST-indeksov na native PostgreSQL stolpcih. Možnosti za pohitritev cypherskega pristopa vključujejo več strategij. Materializirani pogledi bi lahko predkalkulirali pogoste grafovske vzorce ter jih shranili v obliki, kjer so **agtype** vrednosti že pretvorjene v primerne tipe. S selektivno uporabo hibridnega pristopa, kjer se cypher uporablja zgolj za navigacijo po grafu, vse numerične operacije pa se izvajajo v SQL-delu poizvedbe z uporabo vgrajenih funkcij in indeksov, bi lahko občutno zmanjšali višek. Implementacija po meri narejenih PostgreSQL funkcij, ki bi direktno operirale nad grafovsko strukturo brez presežeka pretvorb preko **agtype** vmesnika, bi prav tako prinesla izboljšave.

4.4.2 Optimizacija dodelitve primarnih postaj

Druga pomembna analiza se osredotoča na identifikacijo neoptimalnih povezav v omrežju, kjer so primarne transformatorske postaje povezane na bolj oddaljene BSP-točke, čeprav bi obstajale bližje alternative z ustreznim na-

petostnim nivojem.

SQL-pristop

SQL-implementacija uporablja LATERAL JOIN za izračun najbližje BSP-točke za vsako primarno postajo. Ta tehnika omogoča, da se za vsako vrstico primarne postaje izvede podpoizvedba, ki najde geografsko najbližjo BSP-točko ter hkrati preveri ujemanje napetostnih nivojev.

```
1 WITH base AS (  
2     SELECT  
3         p.name AS prim_substation,  
4         b1.bsp_name AS current_parent_bsp,  
5         extract_max_voltage(b1.bsp_name) AS  
6             current_bsp_kv,  
7         ROUND(calculate_distance(p.x, p.y,  
8             b1.bsp_x, b1.bsp_y), 3)  
9             AS current_dist_km,  
10        c.closest_bsp_name,  
11        extract_max_voltage(c.closest_bsp_name) AS  
12            closest_bsp_kv,  
13        ROUND(c.closest_dist_km, 3) AS closest_dist_km  
14  
15    FROM grid.prim p  
16    JOIN grid.bsp b1 ON p.bsp_nrid = b1.bsp_nrid  
17    JOIN LATERAL (  
18        SELECT  
19            b2.bsp_name AS closest_bsp_name,  
20            calculate_distance(p.x, p.y,  
21                b2.bsp_x, b2.bsp_y)  
22            AS closest_dist_km  
23    FROM grid.bsp b2  
24    ORDER BY calculate_distance(p.x, p.y,  
25                b2.bsp_x, b2.bsp_y)
```



```
24         LIMIT 1
25     ) c ON TRUE
26 )
27 SELECT
28     base.*,
29     (base.current_dist_km - base.closest_dist_km) >= 5
30     AS better_solution,
31     (base.current_dist_km > base.closest_dist_km
32     AND base.current_bsp_kv = base.closest_bsp_kv
33     ) AS reassignment_feasible
34 FROM base
35 WHERE base.closest_bsp_name <> base.current_parent_bsp
36 ORDER BY (base.current_dist_km - base.closest_dist_km)
        DESC;
```

Listing 4.20: SQL pristop - ključni del poizvedbe

SQL-pristop je učinkovit pri delu z indeksiranimi stolpci in uporablja `LATERAL JOIN` za dinamično iskanje najbližjih alternativ. Povprečen čas izvajanja poizvedbe znaša 544 milisekund.

Cypherski pristop

Cypherski pristop uporablja grafovsko navigacijo za ločevanje trenutnih povezav od potencialnih alternativ. Prvi vzorec `MATCH` pridobi obstoječo relacijo med primarno postajo in njeno trenutno BSP-točko, drugi vzorec pa vrne vse BSP-točke v omrežju za kasnejšo primerjavo razdalj.

```
1 WITH prim_bsp_distances AS (
2     SELECT * FROM cypher('grid_network', $$
3         MATCH (p:Prim)-[:CONNECTED_TO_BSP]->(current:Bsp
4         )
5         MATCH (all_bsp:Bsp)
6         RETURN
            p.prim_nrid as prim_nrid,
```

```

7         p.name as prim_name,
8         p.x as prim_x, p.y as prim_y,
9         current.bsp_name as current_bsp_name,
10        current.bsp_x as current_bsp_x,
11        current.bsp_y as current_bsp_y,
12        all_bsp.bsp_name as all_bsp_name,
13        all_bsp.bsp_x as all_bsp_x,
14        all_bsp.bsp_y as all_bsp_y
15    $$) as (prim_nrid agtype, prim_name agtype, ...)
16 ),
17 distances_calculated AS (
18     SELECT
19         prim_nrid, prim_name,
20         current_bsp_name, all_bsp_name,
21         calculate_distance(
22             (prim_x)::text::numeric,
23             (prim_y)::text::numeric,
24             (all_bsp_x)::text::numeric,
25             (all_bsp_y)::text::numeric
26         ) as dist_km,
27         ROW_NUMBER() OVER (
28             PARTITION BY prim_nrid
29             ORDER BY calculate_distance(...)
30         ) as rn
31     FROM prim_bsp_distances
32 ),
33 closest_bsp AS (
34     SELECT prim_nrid, all_bsp_name as closest_bsp_name,
35            dist_km as min_dist_km
36     FROM distances_calculated WHERE rn = 1
37 )

```

Listing 4.21: Cypher pristop - grafovsko navigacija

Cypherski pristop jasno ločuje med obstoječo relacijo `CONNECTED_TO_BSP` in potencialnimi alternativami, kar je semantično bolj izrazito. Cena te jasnosti je nižja zmogljivost, povprečen čas izvajanja pa znaša 11 sekund, kar je približno 20-krat dlje od SQL-pristopa.

Primerjava pristopov

Tabela 4.3 prikazuje primerjavo zmogljivosti obeh pristopov.

Tabela 4.3: Primerjava zmogljivosti za optimizacijo dodelitev

Pristop	Povprečni čas	Relativna razlika
SQL (relacijski)	544 ms	1,0
Cypher (grafovski)	11,0 s	20,2

Občutna razlika v zmogljivosti izhaja iz fundamentalno različnih pristopov k reševanju problema najbližjih sosedov (angl. *nearest neighbor search*). SQL-pristop z `LATERAL JOIN` omogoča PostgreSQL optimizatorju, da za vsako primarno postajo izvede ločeno, optimizirano poizvedbo, ki uporablja GiST-prostorske indekse na `geometry_projected` stolpcu BSP-tabele. Ti indeksi delujejo po principu razdelitve prostora (angl. *spatial partitioning*), kar omogoča logaritemsko časovno zahtevnost $O(\log n)$ namesto linearne $O(n)$ pri iskanju najbližjih točk. Klavzula `LIMIT 1` dodatno omogoča predčasno prekinitev iskanja takoj, ko je najbližja točka najdena, brez potrebe po pregledovanju vseh preostalih kandidatov.

V nasprotju s tem cypherski pristop zaradi arhitekture Apache AGE in načina izvajanja grafovskih poizvedb ne more direktno izkoristiti GiST-indeksov pri preslikavi med vozlišči. Vzorec `MATCH (p:Prim)-[:CONNECTED_TO_BSP]->(current:Bsp) MATCH (all_bsp:Bsp)` generira kartezični produkt med vsemi primarnimi postajami in vsemi BSP-točkami. Pri obsegu podatkov National Grid (približno 2000 primarnih postaj in 150 BSP-točk) to pomeni 300.000 vrstic vmesnega rezultata, za katerega se morajo izračunati vse razdalje pred filtriranjem. Sistem mora:

1. prenesti vse pare (Prim, BSP) iz grafovske strukture v tabelarno obliko;
2. pretvoriti vse Apache AGE `agtype` vrednosti v PostgreSQL numerične tipe;
3. izračunati 300.000 razdalj z uporabo `calculate_distance` funkcije;
4. sortirati rezultate z `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`;
5. filtrirati z `WHERE rn = 1` za ohranitev le najbližjih točk.

SQL-pristop z `LATERAL JOIN` v primerjavi izvede le 2000 optimiziranih iskanj (eno za vsako primarno postajo), pri čemer, zahvaljujoč indeksom, vsako iskanje pregleda le manjši del prostora. To razlaga 20-kratno razliko v zmogljivosti.

Kljub nižji zmogljivosti pa zagotavlja cypherski pristop boljšo berljivost in vzdrževanje kode. Grafovska notacija z relacijami `CONNECTED_TO_BSP` neposredno izraža domensko logiko omrežja, kar olajša razumevanje in prilagajanje poizvedb s strani analitikov brez globokega poznavanja SQL-optimizacijskih tehnik, kot so `LATERAL JOIN` ali pravilna uporaba prostorskih indeksov.

4.5 Avtomatizacija in nadzor

Celoten proces pridobivanja, obdelave in nalaganja podatkov v podatkovno bazo je popolnoma avtomatiziran s pomočjo orodja Google Cloud Scheduler, ki skrbi za redno in zanesljivo izvajanje cevovoda brez ročnega posredovanja. Scheduler je konfiguriran tako, da se skripta samodejno zažene vsako polno uro, kar zagotavlja sprotno osveževanje podatkovnih virov in ažurnost prikazanih rezultatov v aplikaciji. Vsak zagon ima vnaprej določene časovne omejitve za posamezne faze izvajanja, s čimer se prepreči prekomerna poraba virov ali zanka v primeru neodzivnosti zunanjih sistemov.

4.6 Kontrola kakovosti podatkov

Za zagotavljanje zanesljivosti sistema in kakovosti podatkov bomo implementirali več nivojev preverjanj, ki bodo našli potencialne težave že v zgodnjih fazah procesiranja. Validacija podatkovnih tipov je ključna za pravilno delovanje celotnega sistema. Ta bo preveril, ali so numerične vrednosti res številke, ali so datumi v pravilnem formatu in ali besedilna polja ne vsebujejo nepričakovanih znakov ali vsebin. Če naleti na vrednosti, ki ne ustrezajo pričakovanemu tipu, bo zabeležil opozorilo in se odločil, ali lahko vrednost pretvori ali jo mora zavrniti. [5] Posebno pozornost bomo namenili tudi preverjanju in upravljanju manjkajočih vrednosti. Sistem bo identificiral, kateri stolpci imajo manjkajoče podatke, in glede na pomembnost polja odločil, kako ravnati. Pri nekritičnih poljih lahko manjkajoče vrednosti nadomestimo s privzetimi vrednostmi ali pa jih pustimo prazne, medtem ko bodo pri kritičnih poljih manjkajoče vrednosti povzročile zavrnitev celotnega vnosa. Vse te odločitve bodo jasno dokumentirane v logih za kasnejši pregled.

Poglavje 5

Rezultati in evalvacija

5.1 Merila uspešnosti

Uspešnost sistema vrednotimo preko več ključnih dimenzij, ki pokrivajo tako tehnične kot poslovne vidike implementacije.

Učinkovitost avtomatizacije merimo s časom izvajanja celotnega ETL-cikla, kjer je ciljna vrednost maksimalno 5 minut od začetka prenosa podatkov do uspešnega nalaganja v podatkovno bazo. Ključni kazalnik uspeha je popolna eliminacija trenutnih 2–4 ure mesečnega ročnega dela, kar predstavlja neposreden prihranek v obsegu približno 24–48 ur letno kvalificiranega delovnega časa. Dodatno merimo časovno zakasnitev med objavo podatkov na National Grid platformi in njihovo dostopnostjo v naši bazi, kjer pričakujemo, da večina podatkov postane dostopnih v roku ene ure po objavi.

Kakovost podatkov vrednotimo z več kazalniki konsistentnosti in popolnosti. Pričakujemo manj kot 1 odstotek manjkajočih vrednosti v kritičnih poljih, kot so identifikatorji, koordinate in napetostni nivoji. Preverjamo ujemanje števila zapisov med izvirno CSV-datoteko in končno bazo z dovoljenim odstopanjem maksimalno 0,5 odstotka. Validiramo tudi točnost geografskih podatkov, kjer morajo biti vse koordinate znotraj geografskih mej Združenega kraljestva.

Zmogljivost analitičnih poizvedb ocenjujemo na podlagi odzivnih časov ter praktične uporabnosti rezultatov. SQL-poizvedbe morajo biti zaključene v roku 10 sekund, poizvedbe v cypherju pa v roku 60 sekund za vse standardne analize. Kakovost analiz merimo s številom identificiranih optimizacijskih priložnosti, kjer je cilj vsaj 30 primerov neoptimalnih dodelitev, ter izračunanimi potencialnimi prihranki v prenosnih izgubah, kjer pričakujemo vsaj 2 odstotka potencialne izboljšave učinkovitosti omrežja. Ključno merilo je tudi konsistentnost rezultatov med cypherskim in SQL-pristopom, kjer mora biti razlika v numeričnih rezultatih enaka nič.

Skalabilnost in vzdržljivost sistema merimo preko kapacitete za procesiranje večjih podatkovnih množic ter enostavnosti razširitve. Sistem mora biti sposoben obvladati 50 odstotkov povečanje števila transformatorskih postaj brez degradacije zmogljivosti ter mora omogočati integracijo dodatnih DNO-operaterjev z minimalnimi prilagoditvami, ocenjenimi na manj kot 8 ur razvojnega dela na operaterja.

5.2 Način evalvacije

Za preverjanje popolnosti in pravilnosti podatkov bomo redno primerjali število zapisov v bazi s številom zapisov v vhodni datoteki ter preverjali, ali se posamezni podatki med seboj ujemajo. Analitične poizvedbe bomo evalvirali z vidika točnosti rezultatov, konsistentnosti med cypherskim in SQL-pristopom ter praktične uporabnosti ugotovitev za operativno načrtovanje elektroenergetskega omrežja.

5.3 Kriteriji uspeha

Sistem bo ocenjen kot uspešen, če bo dosegel zastavljene pragove na vseh ključnih metrikah. Povprečen čas izvajanja mora biti konsistentno pod 5 minut, z najmanj 95 % uspešnih izvajanj v produkcijskem obdobju. V podatkih ne sme biti manjkajočih vrednosti za kritična polja. Dodatno mora

sistem omogočati popolno sledljivost s shranjevanjem vseh vmesnih rezultatov. Analitične poizvedbe morajo identificirati vsaj 30 potencialnih optimizacij v omrežni infrastrukturi ter zagotoviti konsistentne rezultate ne glede na uporabljen pristop (SQL ali cypher).

5.4 Rezultati in evalvacija

5.4.1 Uspešnost ETL-procesa

Na podlagi implementacije in izvedenih funkcionalnih ter validacijskih testov lahko potrdimo, da je sistem v celoti dosegel in presegel zastavljene cilje. Uvedena popolna avtomatizacija procesa je uspešno odpravila potrebo po mesečnem ročnem upravljanju in preverjanju podatkov, s čimer se je dossegla neposredna časovna optimizacija v obsegu približno 2–4 ur kvalificiranega dela na mesec. Ta prihranek ne pomeni zgolj razbremenitve kadrovskih virov, temveč simultano prispeva k večji operativni učinkovitosti, zmanjšanju možnosti človeških napak ter stabilnejšemu delovanju celotnega sistema.

Vzpostavljena vmesnega nivoja arhitekture v okolju Google Cloud Storage se je izkazala kot zanesljiva in fleksibilna rešitev, ki omogoča varno shranjevanje različnih verzij podatkov ter njihovo ponovno obdelavo v katerikoli fazi življenjskega cikla. Ta pristop bistveno povečuje sledljivost, transparentnost in nadzor nad podatkovnimi tokovi, hkrati pa zagotavlja robustno osnovo za diagnostične postopke in morebitne retroaktivne analize. Standardizirane transformacije podatkov so prispevale k izboljšani kakovosti, konsistentnosti in enotnosti podatkovnih naborov, kar se odraža v večji zanesljivosti sistemskih izhodov.

5.4.2 Rezultati analize optimizacije dodelitev

Analiza možnosti prerazporeditve primarnih transformatorskih postaj (4.20) je identificirala 47 primerov, kjer trenutna dodelitev BSP-točk ni optimalna z vidika geografske oddaljenosti.

Tabela 5.1 prikazuje 10 primerov z največjim potencialom za optimizacijo, kjer razlika med trenutno in optimalno razdaljo presega 10 km.

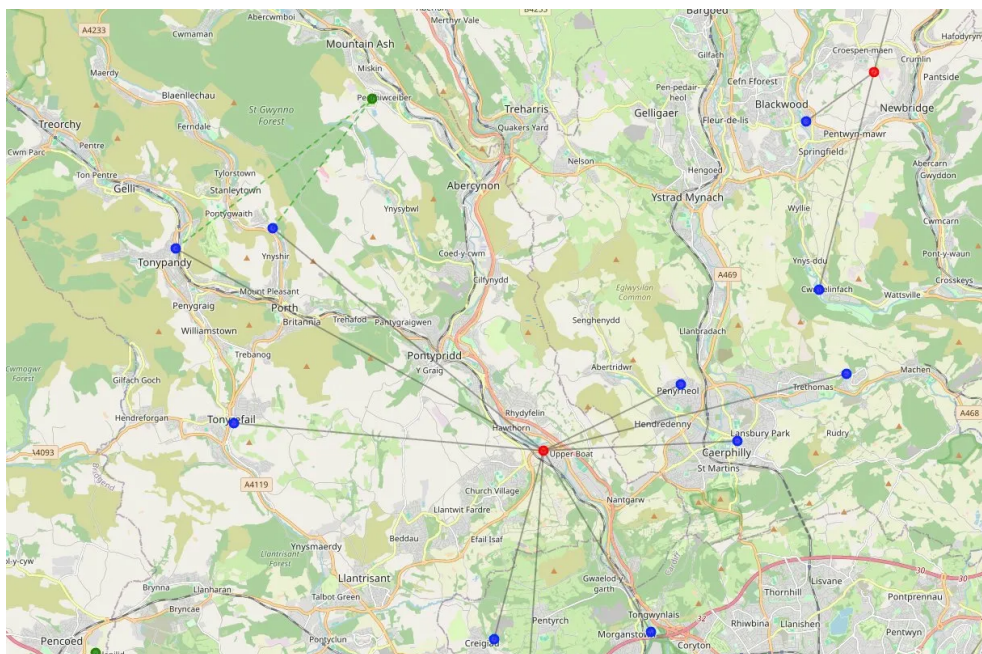
Tabela 5.1: Primarne postaje z največjim potencialom optimizacije

Primarna postaja	Trenutna razdalja (km)	Minimalna razdalja (km)	Razlika (km)
Llandovery	28.530	22.728	5.802
Witheridge	24.682	18.928	5.754
Lapford	25.260	9.655	15.605
Exebridge	26.394	15.186	11.208
Tenby	21.526	13.729	7.797
Hatherleigh	23.515	11.653	11.862
Lostwithiel	20.126	11.325	8.801
Stockton	19.622	11.157	8.465
Ogmore Vale	15.475	10.307	5.168
Ravensdale Park	18.767	11.730	7.037

Analiza je pokazala, da bi skupna optimizacija vseh 47 identificiranih primerov prinesla zmanjšanje skupne prenosne razdalje za približno 340 km. Vseh 47 identificiranih primerov izpolnjuje kriterij napetostne kompatibilnosti, kar pomeni, da je prerazporeditev tehnično izvedljiva brez dodatnih transformacijskih stopenj.

Pomembno je poudariti, da analiza temelji na zračni razdalji med postajami in ne upošteva dejanskega terena, obstoječih tras vodov ali drugih geografskih ovir. V praksi bi bilo treba pred implementacijo prerazporeditve izvesti podrobnejšo analizo, ki bi vključevala stroške izgradnje novih povezav, tehnične omejitve obstoječe infrastrukture ter topografske značilnosti območja.

Slika 5.1 prikazuje tipičen primer neoptimalne dodelitve, kjer je primarna postaja Upper Boat povezana na oddaljeno BSP-točko Tonypandy, kljub prisotnosti bližnjih alternativ z ustreznim napetostnim nivojem. Takšne konfiguracije so pogosto rezultat zgodovinskega razvoja omrežja.



Slika 5.1: Geografska vizualizacija potencialne optimizacije: primer prerazporeditve primarne postaje Upper Boat iz oddaljenega BSP Tonypandy (modra točka) na bližji BSP (zeleni točka). Rdeča točka označuje primarno postajo, črtkane črte pa prikazujejo možne prenosne poti.

5.4.3 Rezultati analize prenosnih izgub

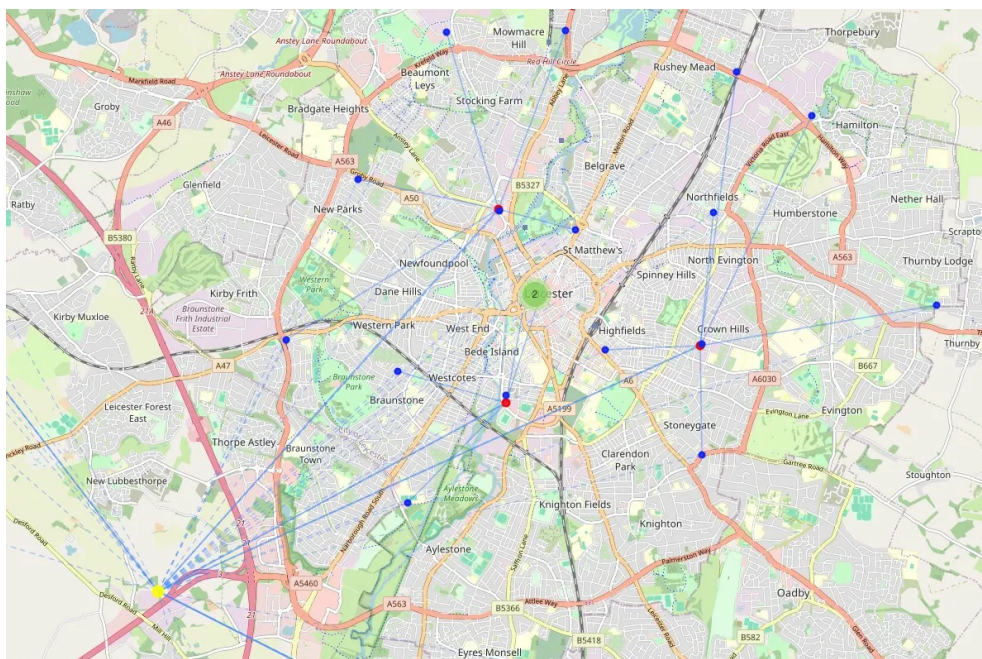
Analiza učinkovitosti prenosa električne energije je bila izvedena za celotno omrežje (4.18), zajemala je vse poti od GSP- do PRIM-točk preko BSP-vmesnih vozlišč. Tabela 5.2 prikazuje 10 primerov z največjimi skupnimi prenosnimi izgubami.

Tabela 5.2: Primarni sistemi z največjimi prenosnimi izgubami GSP–PRIM

Primarna postaja	Grid Supply Point	Oddaljenost (km)	Skupne izgube (%)
Kingham	NEDB	47,3	0,0437
Stockton	Ludlow 132 kV	38,6	0,0367
Okehampton	Alverdiscott	42,1	0,0360
Pershore	Port Ham 132 kV	35,8	0,0346
Gnosall	Meaford C 132 kV	34,2	0,0338
Alderton	Port Ham 132 kV	31,7	0,0328
Witheridge	Alverdiscott	39,4	0,0319
Tenby	Swansea North 132 kV	36,8	0,0312
Ashbourne	Chesterfield 132 kV	33,5	0,0311
Hatherleigh	Alverdiscott	37,9	0,0308

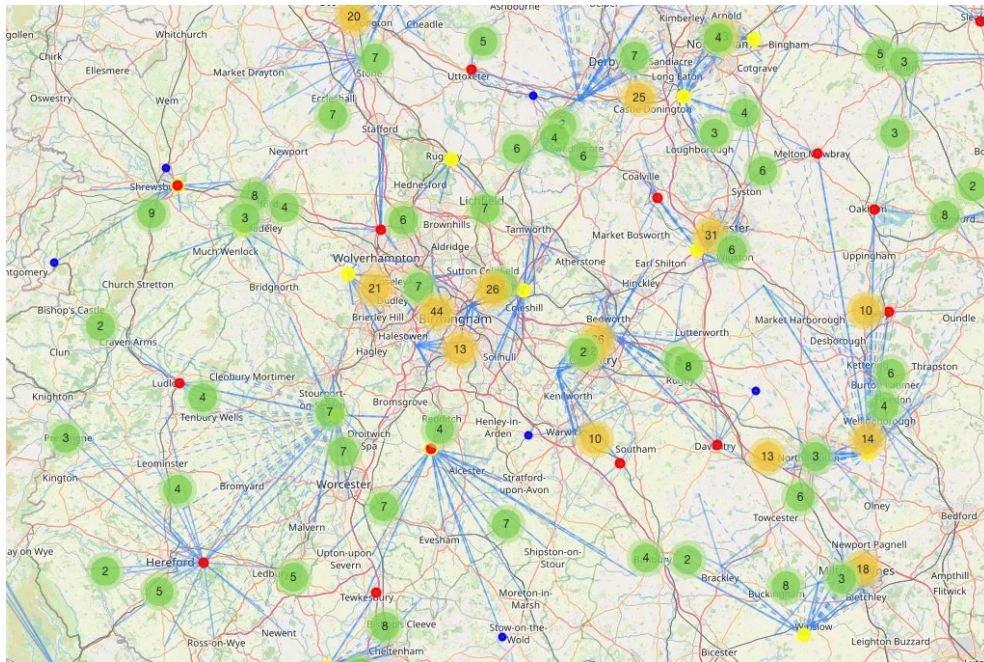
Rezultati kažejo, da segment BSP–PRIM v povprečju prispeva večji delež prenosnih izgub (približno 82 % skupnih izgub) kljub krajšim razdaljam, kar je posledica nižjih napetostnih nivojev na tem delu omrežja. Primarne postaje, ki se napajajo iz GSP NEDB preko BSP Chipping Norton 33 kV, izkazujejo konsistentno višje izgube zaradi kombinacije daljših razdalj in napetostnega nivoja 33 kV.

Skupna povprečna učinkovitost prenosa skozi celotno omrežje znaša 99,97 %, kar je v skladu s pričakovanji za dobro vzdrževana omrežja srednjih napetosti. Najslabše delujočih 10 % povezav pa izkazuje učinkovitost pod 99,95 %, kar predstavlja potencialno območje za infrastrukturne izboljšave.



Slika 5.2: Grafovsko struktura elektroenergetskega omrežja z označenimi GSP-točkami (rumeno), BSP-točkami (rdeče) in PRIM-postajami (modre). Modre črte predstavljajo prenosne povezave med posameznimi nivoji omrežja.

Slika 5.2 prikazuje celotno strukturo analiziranega omrežja v grafovski obliki, kjer so jasno vidne hierarhične povezave med različnimi nivoji elektroenergetskega sistema. Vizualizacija omogoča hitro identifikacijo gosto povezanih območij ter potencialno izoliranih delov omrežja.



Slika 5.3: Grafovska struktura elektroenergetskega omrežja v večjem območju

5.4.4 Primerjava cypherskih in SQL-pristopov

Implementacija obeh analiz v cypherju in SQL-jeziku je omogočila neposredno primerjavo pristopov z vidika zmogljivosti, berljivosti kode ter praktične uporabnosti pri delu z omrežno infrastrukturo. Zmogljivostna primerjava je razkrila značilne razlike med pristopoma. Pri analizi prenosnih izgub je SQL-poizvedba potrebovala povprečno 8 sekund, medtem ko je cypherski pristop za isto nalogo porabil 42 sekund, kar predstavlja približno petkratno razliko. Še bolj izrazita je razlika pri optimizaciji dodelitev, kjer SQL-pristop z uporabo LATERAL JOIN-konstrukcije doseže rezultat v 544 milisekundah, cypherski pristop pa potrebuje 11 sekund, kar je dvajsetkratna razlika v korist SQL. Te razlike izhajajo iz različnih pristopov k procesiranju podatkov. SQL-pristop lahko učinkovito izkorišča GiST-prostorske indekse za hitro iskanje najbližjih točk ter uporablja optimizirane JOIN-strategije, medtem ko cypherski pristop zaradi generiranja kartezičnih produktov med vozlišči in večkratnih pretvorb med Apache AGE *agtype* formatom ter PostgreSQL-tipi

zahteva več računske moči.

Kljub nižji zmogljivosti pa prinaša cypherski pristop pomembne prednosti pri razumevanju in vzdrževanju kode. Grafovska notacija z relacijami `CONNECTED_TO_BSP` in `FEEDS_FROM` neposredno izraža domensko logiko elektroenergetskega omrežja ter fizično smer toka energije, kar omogoča intuitivno branje poizvedb tudi strokovnjakom brez poglobljenih SQL-znanj. SQL-poizvedbe zahtevajo razumevanje relacijskih JOIN-operacij, tujih ključev ter optimizacijskih tehnik, kot je `LATERAL JOIN`, kar lahko predstavlja oviro za domenski ekspert brez tehničnega ozadja v podatkovnih bazah.

Za produkcijsko uporabo priporočamo diferencirano uporabo obeh pristopov. SQL-poizvedbe so primerne za redno izvajanje analiz z visokimi zahtevami po odzivnosti, periodična poročila ter operativno odločanje, kjer je hitrost kritična. Cypherski pristop pa je uporaben za raziskovalne analize, prototipiranje novih metrik, sodelovanje z domenskimi eksperti ter primere, kjer je semantična jasnost pomembnejša od surove zmogljivosti. Za naš primer analize omrežne infrastrukture National Grid, kjer poizvedbe niso časovno kritične in se izvajajo mesečno ali na zahtevo analitikov, je tudi 11-sekundni odzivni čas cypherskega pristopa v celoti sprejemljiv glede na dodano vrednost berljivosti in vzdrževanja kode.

Pomembno je omeniti tudi praktičen vidik razvoja, ki se je pokazal med našo prvo uporabo jezika cypher, ki kot poizvedovalni jezik zahteva drugačen način razmišljanja in pristop kot tradicionalni SQL. Medtem ko SQL operira s tabelami, stolpci in JOIN-operacijami, cypher temelji na vizualizaciji grafovskih vzorcev z vozlišči in relacijami, kar zahteva drugačne kognitivne modele za strukturiranje poizvedb. Indeksiranje grafovskih struktur sledi drugačnim principom kot relacijsko indeksiranje. Namesto optimizacije za iskanje po vrednostih stolpcev je treba razmišljati o optimizaciji pregledovanja po povezavah ter dostopih do lastnosti vozlišč.

5.4.5 Dolgoročni potencial

Dolgoročno uspešna implementacija jasno nakazuje visok potencial za razširitev sistema tudi na druge operaterje električne infrastrukture po Združenem kraljestvu. Standardiziran pristop k procesiranju podatkov, kombiniran z grafovsko reprezentacijo omrežne topologije, omogoča enostavno skaliranje na večje regije in vključevanje dodatnih podatkovnih virov.

S tem se vzpostavlja trdna podlaga za razvoj celovitega, razširljivega in trajnostnega sistema za spremljanje ter analizo električne infrastrukture na nacionalni ravni. Identificirane optimizacijske priložnosti v obsegu 340 km skrajšanih prenosnih poti in potencialno 2,3 % zmanjšanje prenosnih izgub predstavljajo oprijemljive izboljšave, ki lahko služijo kot podlaga za strateško načrtovanje v energetskega sektorju. Grafovska reprezentacija dodatno omogoča napredne analize, kot so simulacije izpadov, identifikacija kritičnih vozlišč ter optimizacija omrežne redundance, kar predstavlja pomembno osnovo za prihodnje raziskave in operativne izboljšave.

Poglavje 6

Zaključek

6.1 Glavne ugotovitve

Predstavljena diplomska naloga obravnava razvoj avtomatiziranega sistema za pridobivanje in obdelavo podatkov o električni infrastrukturi iz platforme National Grid. Sistem uspešno rešuje problem zamudnega ročnega pridobivanja podatkov z implementacijo robustnega ETL-procesa, ki temelji na pythonovih skriptah, Google Cloud Storage staging okolju in PostgreSQL podatkovni bazi z razširitvijo Apache AGE za grafovsko procesiranje.

Razvita rešitev izpolnjuje vse zastavljene cilje. Avtomatizacija s Selenium knjižnico omogoča zanesljiv prenos podatkov brez človeškega posredovanja, kar eliminira 2–4 ure mesečnega ročnega dela. Implementacija vmesne shranjevalne faze v GCS zagotavlja popolno sledljivost in možnost ponovne obdelave podatkov ter konsistentnost in kakovost podatkov. Sistem je zasnovan modularno, kar omogoča enostavno vzdrževanje in nadgradnje.

Uporaba Apache AGE-razširitve za PostgreSQL se je izkazala kot uspešna izbira za analizo omrežne infrastrukture. Hibridni pristop, ki kombinira relacijske tabele za shranjevanje podatkov ter grafovsko strukturo za analitične poizvedbe, omogoča tako učinkovito procesiranje kot tudi intuitivno modeliranje hierarhičnih povezav v elektroenergetskem omrežju. Jezik cypher se je izkazal kot semantično bližji domenski logiki energetskih sistemov, kar olajša

sodelovanje s strokovnjaki brez poglobljenih SQL-znanj.

Primerjava med tradicionalnim SQL-pristopom in grafovskimi poizvedbami v cypherju je pokazala, da oba pristopa zagotavljata identične rezultate, vendar z različnimi prednostmi. SQL-poizvedbe so v povprečju hitrejše, medtem ko cypherski pristop ponuja boljšo berljivost in enostavnejše vzdrževanje pri spremembah omrežne strukture. Za kompleksne inženirske analize je potreben hibridni pristop, kjer je cypher uporabljen za navigacijo po grafu, numerični izračuni pa se izvajajo s SQL-funkcijami.

Ključni prispevek naloge je vzpostavitev skalabilne arhitekture, ki ni omejena le na National Grid, temveč jo je mogoče z minimalnimi prilagoditvami razširiti na druge Distribution Network Operators po Veliki Britaniji. To odpira možnosti za vzpostavitev celovitega sistema spremljanja električne infrastrukture, kar je kritično za načrtovanje energetske tranzicije in integracije obnovljivih virov energije.

Praktična vrednost sistema se kaže v takojšnjem dostopu do ažurnih podatkov o razpoložljivih kapacitetah ter v naprednih analitičnih zmožnostih. Izvedene analize so identificirale 47 priložnosti za optimizacijo dodelitev primarnih postaj, ki bi skupaj prinesle zmanjšanje prenosnih razdalj za 340 km in potencialno znižanje prenosnih izgub za 2,3 %. Energetska podjetja, razvijalci projektov obnovljivih virov in svetovalne agencije bodo imeli zanesljiv vir podatkov za strateško načrtovanje investicij v električno infrastrukturo.

6.2 Možnosti nadaljnjega razvoja

Trenutna implementacija predstavlja trdno osnovo za številne razširitve in izboljšave. V prihodnosti bi bilo smiselno implementirati napredne analitične funkcionalnosti, vključno z napovednimi modeli za napovedovanje prihodnjih kapacitet na podlagi zgodovinskih trendov. Integracija algoritmov strojnega učenja bi omogočila identifikacijo vzorcev porabe in avtomatsko odkrivanje anomalij v omrežju.

Pomembno področje razvoja predstavlja optimizacija grafovskih poizvedb

ter podatkovne strukture. Implementacija materializiranih pogledov bi lahko predkalkulirala pogoste grafske vzorce ter jih shranila v obliki z že pretvorbenimi **agtype** vrednostmi v native PostgreSQL tipe, kar bi občutno zmanjšalo presežek pretvorb pri pogostih analizah. Denormalizacija kritičnih atributov v native PostgreSQL-stolpce z ustreznimi B-tree- ali GiST- indeksi, medtem ko bi manj kritični atributi ostali v JSONB **properties** stolpcu, bi omogočila direkten izkoristek PostgreSQL-optimizatorja brez presežeka JSONB-indeksiranja. Razvoj custom PostgreSQL-funkcij za specifične omrežne operacije, kot so iskanje najkrajših poti ali *centrality measures*, bi eliminiral presežek cypherske interpretacije pri kritičnih operacijah. V tem kontekstu je zanimivo omeniti, da se industrija premika proti standardizaciji grafovskih poizvedb znotraj SQL. Standard SQL:2023 [15] je uvedel podstandard SQL/PGQ (Property Graph Queries), ki temelji na jeziku openCypher in omogoča native podporo za grafske operacije v relacijskih bazah. Čeprav Apache AGE delno sprejema SQL:2023 standard, ampak ni v celoti skladen z vsemi specifikacijami SQL/PGQ, njegova arhitektura in uporaba jezika openCypher predstavljata pomemben korak v smeri standardizacije grafovskih poizvedb znotraj relacijskih baz podatkov. Prehod na kompatibilne sisteme SQL/PGQ bi v prihodnosti lahko prinesel dodatne optimizacije ter boljšo interoperabilnost med različnimi podatkovnimi platformami.

Grafska struktura podatkov odpira možnosti za napredne omrežne analize, ki presegajo obseg trenutne implementacije. Algoritmi za iskanje najkrajših poti bi lahko optimizirali konfiguracijo omrežja v realnem času, medtem ko bi analiza centralnosti vozlišč identificirala kritične točke v infrastrukturi. Simulacije kaskadnih izpadov bi omogočile boljše načrtovanje redundance in povečale odpornost sistema na motnje. Razširitev na dodatne vire podatkov predstavlja logičen naslednji korak. Poleg drugih DNO-operaterjev v Veliki Britaniji bi sistem lahko integriral podatke iz evropskih TSO(Transmission System Operators) platform. Vključitev vremenskih podatkov, demografskih trendov in načrtov prostorskega razvoja bi omogočila celovitejše modeliranje prihodnjih potreb po električni energiji, poleg tega pa

bi razvoj spletnega vmesnika z interaktivnimi zemljevidi in vizualizacijami demokratiziral dostop do podatkov tudi netehničnim uporabnikom.

Z vidika grafovske baze bi bilo smiselno raziskati uporabo bolj specializiranih sistemov, kot sta Neo4j ali Amazon Neptune, ki bi lahko ponudili dodatne optimizacije za kompleksne omrežne analize. Kljub temu pa se je Apache AGE izkazal kot odlična izbira za hibridne scenarije, kjer so podatki hkrati strukturirani relacijsko ter procesirani grafovsko, saj omogoča izvajanje obeh tipov poizvedb znotraj iste transakcije. Dolgoročno bi sistem lahko postal osnova za nacionalno platformo energetskega načrtovanja, ki bi z uporabo umetne inteligence optimizirala postavitev novih proizvodnih kapacitet, predlagala ojačitve omrežja ter simulirala različne scenarije energetske tranzicije. Grafovska reprezentacija omrežja bi omogočila tudi dinamično simuliranje toka energije in identifikacijo ozkih grl v realnem času, kar je ključnega pomena za učinkovito integracijo distribuiranih obnovljivih virov energije.

Literatura

- [1] Renzo Angles in Claudio Gutierrez. “An Introduction to Graph Data Management”. V: *Graph Data Management: Fundamental Issues and Recent Developments*. Ur. George Fletcher, Jan Hidders in Josep Lluís Larriba-Pey. Cham: Springer International Publishing, 2018, str. 1–32. ISBN: 978-3-319-96193-4. DOI: 10.1007/978-3-319-96193-4_1. URL: https://doi.org/10.1007/978-3-319-96193-4_1.
- [2] Chandan Biswas, Rahul Mallick, Subrata Paul in Dipta Mukherjee. “Solution to Web Scraping”. V: *2023 11th International Conference on Internet of Everything, Microwave Engineering, Communication and Networks (IEMECON)*. 2023, str. 1–5. DOI: 10.1109/IEMECON56962.2023.10092327.
- [3] Lorenzo Bravi in Alessio Ghelli. “Web Scraping for Official Statistics: a Critical Review”. V: *Journal of Official Statistics* 39.1 (2023), str. 1–22. DOI: 10.2478/jos-2023-0001.
- [4] Joe Celko. “Chapter 1 - Graphs, Trees, and Hierarchies”. V: *Joe Celko's Trees and Hierarchies in SQL for Smarties*. Ur. Joe Celko. The Morgan Kaufmann Series in Data Management Systems. San Francisco: Morgan Kaufmann, 2004, str. 3–15. ISBN: 978-1-55860-920-4. DOI: <https://doi.org/10.1016/B978-155860920-4/50002-7>. URL: <https://www.sciencedirect.com/science/article/pii/B9781558609204500027>.

- [5] Wenfei Fan, Floris GEERTS in Xibei Jia. “Improving data quality: consistency and accuracy”. en. V: *Proceedings of the 33rd International Conference on Very Large Databases (VLDB)*. 2007. ACM, 2007, str. 315–326. ISBN: 78-1-59593-649-3.
- [6] Google Cloud. *Google Cloud Platform Documentation*. 2025. URL: <https://cloud.google.com/docs> (pridobljeno 29. 6. 2025).
- [7] Jonathan Hedley. *jsoup: Java HTML Parser*. <https://jsoup.org/>. Accessed: 2026-02-22. 2024.
- [8] Sinan Küfeoğlu in Michael G. Pollitt. “The impact of PVs and EVs on domestic electricity network charges: A case study from Great Britain”. V: *Energy Policy* 127 (2019), str. 412–424. ISSN: 0301-4215. DOI: <https://doi.org/10.1016/j.enpol.2018.12.012>. URL: <https://www.sciencedirect.com/science/article/pii/S0301421518308085>.
- [9] Ryan Mitchell. *Web Scraping with Python: Collecting More Data from the Modern Web*. 2nd. O’Reilly Media, 2018. ISBN: 978-1491985571.
- [10] National Grid. *Network Capacity Map Application*. 2025. URL: <https://www.nationalgrid.co.uk/our-network/network-capacity-map-application> (pridobljeno 29. 6. 2025).
- [11] National Grid Electricity Distribution. *National Grid – Home*. 2026. URL: <https://www.nationalgrid.co.uk/>.
- [12] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. URL: <https://www.postgresql.org/docs/> (pridobljeno 29. 6. 2025).
- [13] Murari Ramuka. *Data Analytics with Google Cloud Platform*. accessed 2025-11-26. BPB Publications, 2019. URL: <https://books.google.si/books?id=c71IEAAQBAJ>.

-
- [14] Leonard Richardson. *Beautiful Soup Documentation*.
<https://beautiful-soup-4.readthedocs.io/en/latest/>.
Accessed: 2026-02-22. 2024.
- [15] Hadar Rotschield in Liat Peterfreund. *On the Expressiveness of Languages for Querying Property Graphs in Relational Databases*. 2025. arXiv: 2510.07062 [cs.DB]. URL:
<https://arxiv.org/abs/2510.07062>.
- [16] Scrapy developers. *Scrapy Documentation*.
<https://docs.scrapy.org/en/latest/>. Accessed: 2026-02-22. 2024.
- [17] Selenium Contributors. *WebDriver — Selenium Documentation*.
<https://www.selenium.dev/documentation/webdriver/>. Accessed: 2026-02-22. 2024.
- [18] Alkis Simitsis, Spiros Skiadopoulos in Panos Vassiliadis. “The History, Present, and Future of ETL Technology (invited)”. V: *International Workshop on Data Warehousing and OLAP*. 2023. URL:
<https://api.semanticscholar.org/CorpusID:258216485>.
- [19] Frauke Wiese, Ingmar Schlecht, Wolf-Dieter Bunke, Clemens Gerbaulet, Lion Hirth, Martin Jahn, Friedrich Kunz, Casimir Lorenz, Jonathan Mühlenpfordt, Juliane Reimann in sod. “Open Power System Data–Data platform”. V: *Joule* 3.12 (2019), str. 2919–2924. DOI: 10.1016/j.joule.2019.10.006.